

DSA MASTER HANDBOOK

Arrays + Hashing

Complete FAANG Interview + Competitive Programming Reference

✓ 30 Patterns Covered

✓ Interview Cheat Sheets

✓ FAANG Frequency Tables

✓ C++ Interview Code

✓ Dry Runs + Visualisation

✓ CP Contest Tricks

✓ Complexity Decision Tables

✓ Common Traps + Edge Cases

✓ Pattern Recognition Guide

✓ Learning Roadmap

This handbook is your single source of truth for Arrays and Hashing — the most tested topic in FAANG interviews and Codeforces rounds. Study it, revise it, own it.

TABLE OF CONTENTS

CHAPTER 0 — Fundamentals & Mental Models

- › How to use this handbook
- › Pattern Recognition Framework
- › Constraint Analysis Guide
- › Time Complexity Decision Table
- › Master Mental Models

CHAPTER 1 — Hashing Patterns

- › P-01: Frequency Counting
- › P-02: HashMap Patterns
- › P-03: HashSet Patterns
- › P-04: Greedy + Hashing

CHAPTER 2 — Prefix & Range Patterns

- › P-05: Prefix Sum
- › P-06: Prefix XOR
- › P-07: Prefix/Suffix Arrays
- › P-08: Difference Array

CHAPTER 3 — Window & Pointer Patterns

- › P-09: Fixed Sliding Window
- › P-10: Dynamic Sliding Window
- › P-11: Two Pointer
- › P-12: Dutch National Flag

CHAPTER 4 — Subarray Patterns

- › P-13: Kadane Algorithm
- › P-14: Contribution Technique
- › P-15: Monotonic Stack
- › P-16: Monotonic Queue

CHAPTER 5 — Sorting Patterns

- › P-17: Sorting + Greedy
- › P-18: Bucket Sort
- › P-19: Counting Sort
- › P-20: Cyclic Sort
- › P-21: Binary Search on Arrays

CHAPTER 6 — Voting & Selection Patterns

- › P-22: Boyer-Moore Voting
- › P-23: Merge Intervals
- › P-24: Interval Scheduling
- › P-25: Sweep Line

CHAPTER 7 — Advanced Techniques

- › P-26: Coordinate Compression
- › P-27: Sparse Frequency Tricks
- › P-28: Bitmasking Basics

- › P-29: Rolling Hash Basics
- › P-30: Modular Arithmetic Tricks

CHAPTER 8 — Master Reference Sheets

- › Quick Cheat Sheet — All Patterns
- › FAANG Frequency Master Table
- › Company-wise Pattern Map
- › Pattern Dependency Graph
- › Common Mistakes Bible
- › Interview Communication Guide
- › CP Contest Playbook
- › 30-Day Revision Roadmap

CHAPTER 0 — FUNDAMENTALS & MENTAL MODELS

HOW TO USE THIS HANDBOOK

- Read Pattern Recognition first — knowing signals saves 80% of thinking time
- For each pattern: understand intuition → trace dry run → understand code → memorise complexity
- In interviews: think out loud, name the pattern first, brute → optimal
- For CP: constraint → pattern → code within 30 seconds of reading
- Revise Chapter 8 cheat sheets the night before every interview/contest

CONSTRAINT-DRIVEN PATTERN SELECTION

Read constraints FIRST. They tell you EXACTLY what algorithm to use.

N Constraint	Target Complexity	Likely Pattern	Reason
$N \leq 10$	$O(N!)$ or 2^N	Brute force / Backtrack	Tiny input, any algorithm works
$N \leq 100$	$O(N^2)$ or $O(N^3)$	Nested loops / DP	Quadratic is fine
$N \leq 1,000$	$O(N^2 \log N)$	Sort + Two Pointer	Still manageable
$N \leq 10,000$	$O(N^2)$	Two pointer / HashMap	Avoid $O(N^3)$
$N \leq 100,000$	$O(N \log N)$	Sort/BSearch/PrefixSum	Log factor ok
$N \leq 1,000,000$	$O(N)$ or $O(N \log N)$	Hash / Sliding Window	Must be near-linear
$N \leq 10,000,000$	$O(N)$	Counting sort / Hash	Linear only

PATTERN RECOGNITION FRAMEWORK

Train yourself to ask these questions in order when you see a new problem:

Step	Question to Ask	→ Pattern If YES
1	Are you counting/finding frequency of elements?	Frequency Count / HashMap
2	Does removing duplicates or checking existence matter?	HashSet
3	Are you computing range sums/queries multiple times?	Prefix Sum
4	Do you need XOR over ranges?	Prefix XOR
5	Is there a contiguous subarray condition?	Sliding Window / Kadane
6	Two elements summing/comparing with pointers?	Two Pointer
7	Need max/min over sliding window?	Monotonic Queue
8	Need next greater/smaller element?	Monotonic Stack
9	Range update on array then query?	Difference Array
10	Values too large but relative order matters?	Coordinate Compression
11	Find majority element?	Boyer-Moore Voting
12	Overlapping intervals?	Merge Intervals / Sweep Line
13	Sorted search or binary condition?	Binary Search on Answer

14	Array has values 1..N, and something is missing?	Cyclic Sort / Math
15	Bitwise properties needed?	Bitmask / XOR patterns

MASTER MENTAL MODELS

Memorise these. They make you think like a senior engineer, not a student.

Mental Model	Why It Works / When to Apply
Hashing = O(1) lookup	Replace any O(N) search inside a loop with a hash map. Reduces $O(N^2) \rightarrow O(N)$.
Prefix Sum = Range query in O(1)	Precompute cumulative array. $\text{sum}(L,R) = \text{pre}[R] - \text{pre}[L-1]$. Works for any associative operation.
Sliding Window = Moving frame on array	Don't recompute the whole window. Add new element, remove old. State stays valid at all times.
Two Pointer = Sorted array shortcut	When array is sorted, two pointers can find pairs in O(N) instead of $O(N^2)$.
Monotonic Stack = Remember useful past	Keep elements in order. When you add new element, pop everything smaller/larger. Solves 'next greater element' class.
Frequency Map = Compress information	Count occurrences in O(N). Answer queries about counts/modes/unique in O(1).
Difference Array = Bulk range updates	Instead of updating each index in a range, update only endpoints. O(1) update, O(N) to reconstruct.
Sorting reveals structure	Once sorted, equal elements are adjacent. Binary search works. Two pointers work. Greedy often works.
Contribution Technique = Flip the question	Instead of asking 'what does this subarray contribute', ask 'how many subarrays does this element contribute to'.
XOR is self-inverse	$a \oplus a = 0$. XOR all elements, duplicates cancel. Perfect for finding single/missing elements.

FREQUENCY COUNTING

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★■

Companies: Google, Amazon, Meta, Microsoft, Adobe, Bloomberg, Uber

RECOGNITION SIGNALS

- Problem asks for count, frequency, or occurrence of elements
- Words: 'most frequent', 'appears k times', 'unique elements', 'distinct count'
- Any time you need to 'group' elements by value
- Anagram / permutation problems (same char counts)
- Top-K elements by frequency

INTUITION

You have elements and you need to KNOW HOW MANY TIMES each appeared. Instead of scanning the whole array every time, do ONE pass to count everything. Then answer queries in $O(1)$. Think of it as building a frequency register first.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N \cdot M)$	$O(1)$	Learning / tiny N
Optimal	$O(N)$	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Brute Force: For each element, scan entire array to count
// Time:  $O(N^2)$ , Space:  $O(1)$ 
int countOccurrence(vector<int>& arr, int target) {
    int count = 0;
    for (int x : arr) //  $O(N)$ 
        if (x == target) count++; // scanning for each query =  $O(N)$  per query
    return count;
}
// If M queries:  $O(N * M)$  – terrible for large inputs
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Optimal: Build frequency map in ONE pass, answer in  $O(1)$ 
// Time:  $O(N)$ , Space:  $O(N)$ 
```

[illegible]

```
vector<int> topKFrequent(vector<int>& nums, int k) {
```

```
unordered_map<int,int> freq;
```

```
for (int x : nums) freq[x]++; // O(N): build frequency map
```

```
// Min-heap of size K: keeps top K frequent
```

```
priority_queue<pair<int,int>, vector<pair<int,int>>,
```

```
greater<pair<int,int>>> minHeap;
```

```
for (auto& [val, cnt] : freq) {
```

```
minHeap.push({cnt, val});
```

```
if ((int)minHeap.size() > k) minHeap.pop(); // evict least frequent
```

}

```
vector<int> result;
```

```
while (!minHeap.empty()) {
```

```
result.push_back(minHeap.top().second);
```

```
minHeap.pop();
```

}

```
return result; // O(N log K) total
```

}

[illegible]

```
bool isAnagram(string s, string t) {
```

```
if (s.size() != t.size()) return false;
```

```
unordered_map<char,int> freq;
```

```
for (char c : s) freq[c]++;
```

```
for (char c : t) {
```

```
freq[c]--;
```

```
if (freq[c] < 0) return false; // t has extra char
```

}

```
return true;
```

}

DRY RUN — nums=[1,1,1,2,2,3], k=2

Step	Action	freq map	heap	Result
Build freq	Count all	{1:3,2:2,3:1}	—	—
Add (3,1)	Push to heap	{1:3,2:2,3:1}	[(1,3)]	—
Add (2,2)	Push to heap	{1:3,2:2,3:1}	[(1,3),(2,2)]	—
Add (1,3)	Push, size=3>k=2	{1:3,2:2,3:1}	[(2,2),(3,1)]	—
Pop min	Pop (1,3)	{1:3,2:2,3:1}	[(2,2),(3,1)]	[1,2]

COMMON MISTAKES + EDGE CASES

- ✗ Using map instead of unordered_map — adds unnecessary $O(\log N)$ per operation
- ✗ Forgetting to handle empty input or $k > \text{unique elements}$
- ✗ Integer overflow when frequency can be very large
- ✗ For character frequency: use `int[26]` array instead of hash map — much faster
- ✗ Not checking if both strings have same length before anagram check

INTERVIEW TIPS

- ✓ Always say: 'I'll use a frequency map to count in $O(N)$ first'
- ✓ For Top-K: mention bucket sort approach $O(N)$ as follow-up
- ✓ For fixed alphabet (a-z): prefer `int[26]` over unordered_map — $O(1)$ space
- ✓ Follow-up: 'What if stream is infinite?' → use min-heap of size K
- ✓ Always clarify: case-sensitive? Unicode or ASCII only?

CP CONTEST TRICKS

- ★ For values $1..N$: use `int` array `freq[N+1]` instead of map — 10x faster
- ★ Bucket sort on frequencies: place elements in buckets by count → $O(N)$ top-K
- ★ Coordinate compress large values before frequency counting
- ★ For anagram groups: sort each string as key → unordered_map
- ★ Frequency of frequency: `freq_of_freq[f]` = how many elements appear exactly f times

SIMILAR PROBLEMS

Problem	Platform	Relation
Two Sum	LeetCode 1	HashMap stores complement → $O(N)$
Top K Frequent Elements	LeetCode 347	Frequency map + heap or bucket sort
Valid Anagram	LeetCode 242	Char frequency comparison
Group Anagrams	LeetCode 49	Sorted string as key in HashMap
First Unique Character	LeetCode 387	Frequency map then scan
Ransom Note	LeetCode 383	Char frequency subtraction
Subarray Sum Equals K	LeetCode 560	Prefix sum + frequency map
Longest Consecutive Sequence	LeetCode 128	HashSet for $O(1)$ lookup

HASHMAP PATTERNS

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★■

Companies: Google, Meta, Amazon, Microsoft, Apple, Airbnb, Stripe

RECOGNITION SIGNALS

- Need to find if complement/pair exists: 'two sum', 'four sum'
- Need to remember something from the past while scanning
- Words: 'find pair', 'exists', 'mapping', 'cache', 'seen before'
- Reduce $O(N^2)$ pair search to $O(N)$
- Need index of first occurrence

INTUITION

HashMap answers: 'Have I seen X before?' or 'Where did I see X?' in $O(1)$. Classic trick: for Two Sum, instead of checking all pairs, for each element ask 'is (target - element) in my map?' You're trading space for time — pay $O(N)$ memory to avoid $O(N^2)$ work.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N^2)$	$O(1)$	Learning / tiny N
Optimal	$O(N)$	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Two Sum brute force: check all pairs
// Time:  $O(N^2)$ , Space:  $O(1)$ 
vector<int> twoSum_brute(vector<int>& nums, int target) {
    int n = nums.size();
    for (int i = 0; i < n; i++)
        for (int j = i+1; j < n; j++)
            if (nums[i] + nums[j] == target)
                return {i, j};
    return {};
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Two Sum optimal: HashMap for  $O(1)$  complement lookup
```

```

// Time: O(N), Space: O(N)
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int,int> seen; // value → index
    for (int i = 0; i < (int)nums.size(); i++) {
        int complement = target - nums[i];
        if (seen.count(complement))
            return {seen[complement], i};
        seen[nums[i]] = i; // store AFTER check (avoid using same element twice)
    }
    return {};
}

// Longest Subarray with Sum K: prefix sum + HashMap
int subarraySum(vector<int>& nums, int k) {
    unordered_map<int,int> prefixCount;
    prefixCount[0] = 1; // empty prefix
    int sum = 0, count = 0;
    for (int x : nums) {
        sum += x;
        count += prefixCount[sum - k]; // subarrays ending here with sum = k
        prefixCount[sum]++;
    }
    return count;
}

// Longest subarray with equal 0s and 1s
int findMaxLength(vector<int>& nums) {
    unordered_map<int,int> firstSeen;
    firstSeen[0] = -1;
    int balance = 0, maxLen = 0;
    for (int i = 0; i < (int)nums.size(); i++) {
        balance += (nums[i] == 1) ? 1 : -1;
        if (firstSeen.count(balance))
            maxLen = max(maxLen, i - firstSeen[balance]);
        else
            firstSeen[balance] = i;
    }
    return maxLen;
}

```

DRY RUN — Two Sum: nums=[2,7,11,15], target=9

i	nums[i]	complement=9-x	seen map	Found?
0	2	7	{}	No → add {2:0}
1	7	2	{2:0}	YES! → return [0,1]

KEY HASHMAP PATTERNS TO MEMORISE

- **Two Sum pattern**: store complement, check existence → $O(N)$
- **Prefix sum + map**: $\text{sum}(L,R)=k \rightarrow$ check if $(\text{currentSum}-k)$ exists in map
- **Balance technique**: replace $0 \rightarrow 1$, track running sum, find longest zero-sum subarray
- **Index mapping**: $\text{map}[\text{value}] = \text{firstIndex}$, answer is $i - \text{map}[\text{value}]$
- **Grouping**: $\text{map}[\text{key}].\text{push_back}(\text{element})$ for grouping anagrams, etc.

COMMON MISTAKES + EDGE CASES

- ✗ Storing index **BEFORE** checking complement — can use same element twice
- ✗ Using int overflow as key when values can be large
- ✗ Forgetting edge case: $\text{map}[0] = -1$ for prefix sum problems (empty prefix)
- ✗ map vs unordered_map: map is $O(\log N)$, unordered_map is $O(1)$ amortised
- ✗ Not handling multiple valid answers when problem guarantees exactly one

INTERVIEW TIPS

- ✓ Say: 'I'll use a HashMap to get $O(1)$ lookup, reducing overall to $O(N)$ '
- ✓ Always check: can same element be used twice? If no, add to map **AFTER** check
- ✓ Follow-up: sorted array? Use two pointers instead — $O(N)$ time, $O(1)$ space
- ✓ Follow-up: all integers? Can use array if range is bounded
- ✓ Explain the 'complement idea' clearly — most interviewers love this insight

CP CONTEST TRICKS

- ★ Prefix sum + map handles: subarray sum = K, equal 0s and 1s, longest subarray
- ★ When values are small ($1..1e6$): use array instead of unordered_map — 5x faster
- ★ Reserve hash map capacity: $\text{freq.reserve}(n)$; $\text{freq.max_load_factor}(0.25)$ → faster
- ★ Custom hash for pairs: use hash combine trick to avoid collisions
- ★ For string problems: polynomial rolling hash converts string key to int key

SIMILAR PROBLEMS

Problem	Platform	Relation
Two Sum	LeetCode 1	Core HashMap pattern
Subarray Sum Equals K	LeetCode 560	Prefix sum + HashMap
Contiguous Array	LeetCode 525	Balance technique ($0 \rightarrow -1$)
Longest Subarray with Sum K	GFG	Prefix sum + first index map
4Sum II	LeetCode 454	Two HashMaps for pair sums
Pairs with difference K	LeetCode 532	HashMap existence check
Max Points on a Line	LeetCode 149	Slope HashMap
Group Anagrams	LeetCode 49	Sorted key HashMap

HASHSET PATTERNS

FAANG FREQ

★★★★■

LEETCODE

★★★★★

CODEFORCES

★★★██

Companies: Google, Amazon, Meta, LinkedIn, Palantir

RECOGNITION SIGNALS

- Need to check if element was seen before (existence, not count)
- Need to detect duplicates or cycles
- Words: 'contains duplicate', 'unique', 'no repeats', 'distinct'
- Longest streak without repeating characters/numbers
- Check if array is a permutation of 1..N

INTUITION

HashSet = 'has this appeared before?' in $O(1)$. Use it when you DON'T need counts, only existence. Longest Consecutive Sequence: put all numbers in set, then for each number check if it's the START of a sequence (i.e., $\text{num}-1$ not in set), then count how long the streak goes.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N^2)$	$O(1)$	Learning / tiny N
Optimal	$O(N)$	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Contains Duplicate brute force: check all pairs
// Time:  $O(N^2)$ , Space:  $O(1)$ 
bool containsDuplicate_brute(vector<int>& nums) {
    int n = nums.size();
    for (int i = 0; i < n; i++)
        for (int j = i+1; j < n; j++)
            if (nums[i] == nums[j]) return true;
    return false;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Contains Duplicate: HashSet  $O(N)$ 
bool containsDuplicate(vector<int>& nums) {
```

```

unordered_set<int> seen;
for (int x : nums) {
    if (seen.count(x)) return true;
    seen.insert(x);
}
return false;
}

// Longest Consecutive Sequence (LeetCode 128) – O(N)
int longestConsecutive(vector<int>& nums) {
    unordered_set<int> numSet(nums.begin(), nums.end());
    int maxLen = 0;
    for (int num : numSet) {
        if (!numSet.count(num - 1)) { // num is sequence START
            int curr = num, streak = 1;
            while (numSet.count(curr + 1)) { curr++; streak++; }
            maxLen = max(maxLen, streak);
        }
    }
    return maxLen;
}

// Happy Number (LeetCode 202) – cycle detection with set
bool isHappy(int n) {
    unordered_set<int> seen;
    while (n != 1) {
        if (seen.count(n)) return false; // cycle detected
        seen.insert(n);
        int sum = 0;
        while (n > 0) { int d = n%10; sum += d*d; n /= 10; }
        n = sum;
    }
    return true;
}

```

DRY RUN — Longest Consecutive: [100,4,200,1,3,2]

num	num-1 in set?	Action	streak	maxLen
100	99? No	Start: 100→101? No	1	1
4	3? Yes	Skip (not start)	—	1
200	199? No	Start: 200→201? No	1	1
1	0? No	Start: 1→2→3→4→5? No	4	4
3	2? Yes	Skip	—	4
2	1? Yes	Skip	—	4

COMMON MISTAKES + EDGE CASES

- ✗ Starting count for every element — key insight: only start from sequence beginnings
- ✗ Not building set from array first — the two-pass approach is critical
- ✗ Using sorted approach ($O(N \log N)$) when $O(N)$ HashSet works
- ✗ Forgetting that `unordered_set` has $O(N)$ worst-case due to hash collisions
- ✗ Using `set` ($O(\log N)$) when `unordered_set` ($O(1)$) is available

INTERVIEW TIPS

- ✓ Longest Consecutive: always explain the 'only start from sequence head' insight
- ✓ For cycle detection: HashSet works but Floyd's algorithm is $O(1)$ space — mention both
- ✓ Clarify: is the array sorted? If yes, sorting + linear scan beats HashSet
- ✓ Follow-up: find the MISSING number → can do XOR or math instead of HashSet
- ✓ Mention: for small value range ($1..N$), can use boolean array instead of HashSet

CP CONTEST TRICKS

- ★ For 'has this value been seen' checks in CF problems: bitset is fastest for $0..N$
- ★ Avoid TLE: use `unordered_set` with custom hash for pairs/strings
- ★ Bitset for existence of values $0..N$: 8x memory efficient + fast
- ★ For permutation check ($1..N$): sum and XOR both present → $O(1)$ space trick
- ★ Two-pointer on sorted array often replaces HashSet with $O(1)$ extra space

SIMILAR PROBLEMS

Problem	Platform	Relation
Contains Duplicate	LeetCode 217	Basic HashSet existence check
Longest Consecutive Sequence	LeetCode 128	Sequence start detection
Happy Number	LeetCode 202	Cycle detection with set
Single Number	LeetCode 136	XOR trick — no extra space
Intersection of Two Arrays	LeetCode 349	Two HashSets
Missing Number	LeetCode 268	Math/XOR trick over HashSet
Find Duplicate Number	LeetCode 287	Floyd's cycle or HashSet
Word Pattern	LeetCode 290	Bijection with two HashMaps

PREFIX SUM

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★★

Companies: Google, Meta, Amazon, Goldman Sachs, Bloomberg, Adobe, Oracle

RECOGNITION SIGNALS

- Multiple range sum queries on static array
- Subarray sum equals K (with HashMap)
- Words: 'sum of subarray', 'range sum', 'cumulative sum'
- 2D matrix range sum queries
- Count of subarrays with property based on sum

INTUITION

Naive range sum: $\text{sum}(L,R)$ costs $O(N)$ per query. Observation: $\text{sum}(L,R) = \text{sum}(0,R) - \text{sum}(0,L-1)$. Pre-build cumulative sums in $O(N)$. Now every query is $O(1)$. This pattern converts 'repeated range work' into 'one-time precomputation'.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N)$ per query	$O(1)$	Learning / tiny N
Better	$O(N)$ build	$O(N)$	When multiple queries needed
Optimal	$O(N)$ build, $O(1)$ query	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Brute: recompute sum for each query
// Time: O(N) per query = O(N*Q) total
int rangeSum_brute(vector<int>& arr, int L, int R) {
    int sum = 0;
    for (int i = L; i <= R; i++) sum += arr[i]; // O(N) per query
    return sum;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Prefix Sum: O(N) build, O(1) per query
class PrefixSum {
    vector<long long> pre;
```

```

public:
PrefixSum(vector<int>& arr) {
int n = arr.size();
pre.resize(n + 1, 0);
for (int i = 0; i < n; i++)
pre[i+1] = pre[i] + arr[i]; // pre[i] = sum of first i elements
}

long long query(int L, int R) { // 0-indexed [L, R]
return pre[R+1] - pre[L];
}

};

// Subarray Sum Equals K (LeetCode 560) – prefix sum + hashmap
int subarraySum(vector<int>& nums, int k) {
unordered_map<int,int> prefixCount{{0, 1}}; // sum=0 seen once (empty prefix)
int sum = 0, count = 0;
for (int x : nums) {
sum += x;
count += prefixCount[sum - k]; // how many times (sum-k) appeared before
prefixCount[sum]++;
}
return count;
}

// 2D prefix sum (matrix range queries)
vector<vector<int>> build2D(vector<vector<int>>& mat) {
int m = mat.size(), n = mat[0].size();
vector<vector<int>> pre(m+1, vector<int>(n+1, 0));
for (int i = 1; i <= m; i++)
for (int j = 1; j <= n; j++)
pre[i][j] = mat[i-1][j-1] + pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1];
return pre;
}

int query2D(vector<vector<int>>& pre, int r1,int c1,int r2,int c2) {
return pre[r2+1][c2+1] - pre[r1][c2+1] - pre[r2+1][c1] + pre[r1][c1];
}

```

DRY RUN — arr=[3,1,4,1,5], queries: sum(1,3), sum(0,4)

i	arr[i]	pre[i+1]	Query	Answer
0	3	3	—	—
1	1	4	—	—
2	4	8	sum(1,3)=pre[4]-pre[1]	8-3=5 ✓
3	1	9	—	—
4	5	14	sum(0,4)=pre[5]-pre[0]	14-0=14 ✓

ADVANCED: SUBARRAY SUM = K DRY RUN

nums=[1,1,1], k=2 → Expected: 2

i	x	sum	sum-k	prefixCount[sum-k]	count	map update
—	—	0	—	—	0	{0:1}
0	1	1	-1	0	0	{0:1,1:1}
1	1	2	0	1	1	{0:1,1:1,2:1}
2	1	3	1	1	2	{0:1,1:1,2:1,3:1}

COMMON MISTAKES + EDGE CASES

✗ Off-by-one: $\text{pre}[i]$ = sum of FIRST i elements (not index i). $\text{pre}[0]=0$ always.

✗ Forgetting $\text{pre}[0]=1$ in prefix count map for subarray sum = K

✗ Not using long long for large sums (overflow with $N=1e5$, values= $1e9$)

✗ 2D prefix sum formula: inclusion-exclusion, easy to mess up signs

✗ Range sum formula: $\text{sum}(L,R) = \text{pre}[R+1] - \text{pre}[L]$, not $\text{pre}[R] - \text{pre}[L-1]$

INTERVIEW TIPS

✓ Always start with: 'I'll precompute prefix sums in $O(N)$ to answer queries in $O(1)$ '

✓ For subarray sum = K: explain prefix sum + map as 'counting how often we've seen sum-k before'

✓ Common follow-up: what if array is modified? Then use Fenwick Tree (BIT)

✓ 2D prefix sum is high-value follow-up — learn the inclusion-exclusion formula

✓ Mention: prefix sum is special case of prefix operation (works for XOR, product too)

CP CONTEST TRICKS

★ $\text{Pre}[]$ size = $N+1$ with $\text{pre}[0]=0$ avoids boundary checks — always do this

★ Prefix XOR: $\text{pre}[R+1] \text{ XOR } \text{pre}[L] = \text{XOR of arr}[L..R]$

★ Prefix MAX/MIN: useful for 'is there any element $> X$ in range?' with binary search

★ Offline queries: sort queries, process with prefix sum

★ 2D prefix sum: essential for CF problems with matrix queries

SIMILAR PROBLEMS

Problem	Platform	Relation
Range Sum Query	LeetCode 303	Direct prefix sum application
Subarray Sum Equals K	LeetCode 560	Prefix sum + HashMap
Product of Array Except Self	LeetCode 238	Prefix + suffix product
Find Pivot Index	LeetCode 724	Left prefix = right suffix sum
Number of Ways to Split Array	LeetCode 2270	Prefix sum + condition
Matrix Block Sum	LeetCode 1314	2D prefix sum
Count Submatrices with Top-Left Element	LeetCode 1504	2D prefix
Max Sum of Rectangle No Larger Than K	LeetCode 363	2D prefix + BST

PREFIX XOR

FAANG FREQ

★★★★■

LEETCODE

★★★★■

CODEFORCES

★★★★★

Companies: Google, Codeforces heavy, Amazon, Microsoft

RECOGNITION SIGNALS

- XOR of subarray: 'XOR of elements from L to R'
- Find element appearing odd/even number of times
- Missing or duplicate in range 1..N
- Problems involving XOR with range queries
- Bitwise toggling patterns

INTUITION

XOR has a magic property: $a \oplus a = 0$. So if you XOR a number with itself, it cancels. Prefix XOR works like prefix sum but for XOR. $\text{XOR}(L, R) = \text{preXOR}[R+1] \oplus \text{preXOR}[L]$. For 'find single number': XOR all elements, pairs cancel, lone number remains.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N)$ per query	$O(1)$	Learning / tiny N
Optimal	$O(N)$	$O(N)$ for prefix, $O(1)$ for single-pass	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// XOR of range: brute force
// Time: O(N) per query
int rangeXOR_brute(vector<int>& arr, int L, int R) {
    int xorVal = 0;
    for (int i = L; i <= R; i++) xorVal ^= arr[i];
    return xorVal;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Prefix XOR: O(N) build, O(1) query
vector<int> buildPrefixXOR(vector<int>& arr) {
```

```

int n = arr.size();
vector<int> preXOR(n + 1, 0);
for (int i = 0; i < n; i++)
preXOR[i+1] = preXOR[i] ^ arr[i];
return preXOR;
}

int rangeXOR(vector<int>& preXOR, int L, int R) {
return preXOR[R+1] ^ preXOR[L]; // XOR of arr[L..R]
}

// Single Number (LeetCode 136): XOR all, pairs cancel
int singleNumber(vector<int>& nums) {
int result = 0;
for (int x : nums) result ^= x;
return result;
}

// Missing Number (LeetCode 268): XOR with expected 0..N
int missingNumber(vector<int>& nums) {
int n = nums.size(), xorAll = 0;
for (int i = 0; i <= n; i++) xorAll ^= i; // XOR 0..N
for (int x : nums) xorAll ^= x; // XOR actual values
return xorAll; // remaining = missing number
}

// Find two non-repeating numbers (XOR trick)
vector<int> findTwoSingle(vector<int>& nums) {
int xorAll = 0;
for (int x : nums) xorAll ^= x; // xorAll = a XOR b
int diffBit = xorAll & (-xorAll); // lowest set bit (a and b differ here)
int a = 0, b = 0;
for (int x : nums) {
if (x & diffBit) a ^= x; // group 1
else b ^= x; // group 2
}
return {a, b};
}

```

DRY RUN — Single Number: [4,1,2,1,2]

Step	Current num	XOR result	Reason
Start	—	0	Initial value
1	4	4	0 XOR 4 = 4
2	1	5	4 XOR 1 = 5
3	2	7	5 XOR 2 = 7

4	1	6	7 XOR 1 = 6 (1 cancels)
5	2	4	6 XOR 2 = 4 (2 cancels) → ANS=4

COMMON MISTAKES + EDGE CASES

- ✗ Confusing XOR with OR/AND — XOR flips bits, pairs cancel to 0
- ✗ Not knowing: $a \text{ XOR } a = 0$, $a \text{ XOR } 0 = a$, XOR is commutative and associative
- ✗ For two non-repeating numbers: forgetting to split groups by diffBit
- ✗ Prefix XOR off-by-one: same indexing issue as prefix sum — use size $N+1$

INTERVIEW TIPS

- ✓ XOR trick is an elegant $O(1)$ space solution — interviewers love it
- ✓ Say: 'I'll use the XOR self-inverse property: $a \text{ XOR } a = 0$ '
- ✓ For two missing/extra: explain the 'split by diffBit' technique clearly
- ✓ XOR is great for follow-ups when interviewer asks for $O(1)$ space
- ✓ Mention: XOR works for 'appears odd number of times' problems generally

CP CONTEST TRICKS

- ★ Prefix XOR on competitive problems: faster than segment trees for XOR range queries
- ★ XOR basis (linear basis): find linearly independent XOR subset — advanced CF trick
- ★ For toggle problems: XOR with 1 flips $0 \leftrightarrow 1$
- ★ XOR swap: $a^b; b^a; a^b$ — avoid in interviews (confusing), mention only if asked
- ★ XOR from 1 to N has pattern: $[N, 1, N+1, 0]$ for $N\%4 = [0,1,2,3]$ — memorise!

SIMILAR PROBLEMS

Problem	Platform	Relation
Single Number	LeetCode 136	XOR all elements
Single Number II	LeetCode 137	Bit counting for $3n+1$
Single Number III	LeetCode 260	Two singles via diffBit split
Missing Number	LeetCode 268	XOR with expected range
Find Duplicate	LeetCode 287	Floyd's cycle or XOR
XOR Queries of Subarray	LeetCode 1310	Direct prefix XOR
Decode XOR-encoded array	LeetCode 1720	Reverse XOR encoding
Maximum XOR of Two Numbers	LeetCode 421	Trie on XOR prefix

FIXED SLIDING WINDOW

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★■

Companies: Amazon, Google, Meta, Microsoft, Uber, PayPal

RECOGNITION SIGNALS

- Subarray/substring of FIXED size K is given
- Find max/min/average/sum of all windows of size K
- Words: 'window of size k', 'consecutive k elements'
- First negative in every window, max of every window of size K
- Any aggregate over all subarrays of fixed length

INTUITION

You have a window of fixed size K sliding across the array. Naive: recompute from scratch for each window = $O(N \cdot K)$.
 Insight: when window moves right by 1, only ONE element enters and ONE leaves. Just update: $\text{new_sum} = \text{old_sum} - \text{arr}[\text{left}] + \text{arr}[\text{right}]$. $O(N)$ total.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N \cdot K)$	$O(1)$	Learning / tiny N
Optimal	$O(N)$	$O(1)$ or $O(K)$ for freq map	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Brute: recompute sum for each window
// Time: O(N*K), Space: O(1)
vector<double> slidingWindowAvg_brute(vector<int>& nums, int k) {
    vector<double> result;
    int n = nums.size();
    for (int i = 0; i <= n - k; i++) {
        double sum = 0;
        for (int j = i; j < i + k; j++) sum += nums[j]; // O(K) per window
        result.push_back(sum / k);
    }
    return result;
}
```


OPTIMAL SOLUTION (Interview-ready C++)

```
// Fixed Window: maintain running sum, O(N)
vector<double> movingAverage(vector<int>& nums, int k) {
    vector<double> result;
    int n = nums.size();
    double windowSum = 0;

    for (int i = 0; i < n; i++) {
        windowSum += nums[i]; // add new element
        if (i >= k) windowSum -= nums[i - k]; // remove element leaving window
        if (i >= k - 1) // window is full
            result.push_back(windowSum / k);
    }
    return result;
}

// Maximum sum of subarray of size K
int maxSumSubarrayK(vector<int>& arr, int k) {
    int n = arr.size(), windowSum = 0, maxSum = INT_MIN;
    for (int i = 0; i < n; i++) {
        windowSum += arr[i];
        if (i >= k) windowSum -= arr[i - k];
        if (i >= k - 1) maxSum = max(maxSum, windowSum);
    }
    return maxSum;
}

// Count distinct elements in every window of size K
// Use freq map: if freq[x] becomes 0, remove from map
vector<int> countDistinct(vector<int>& arr, int k) {
    vector<int> result;
    unordered_map<int,int> freq;
    for (int i = 0; i < (int)arr.size(); i++) {
        freq[arr[i]]++;
        if (i >= k) {
            freq[arr[i-k]]--;
            if (freq[arr[i-k]] == 0) freq.erase(arr[i-k]);
        }
        if (i >= k-1) result.push_back(freq.size());
    }
    return result;
}
```

DRY RUN — maxSum K=3: arr=[2,1,5,1,3,2]

i	arr[i]	Add	Remove	windowSum	maxSum
---	--------	-----	--------	-----------	--------

0	2	+2	—	2	—
1	1	+1	—	3	—
2	5	+5	—	8	8
3	1	+1	-arr[0]=2	7	8
4	3	+3	-arr[1]=1	9	9
5	2	+2	-arr[2]=5	6	9

COMMON MISTAKES + EDGE CASES

✗ Window is full condition: check $i \geq k-1$, not $i \geq k$

✗ Remove condition: subtract $arr[i-k]$, not $arr[i-k-1]$

✗ Integer overflow for large values and large K — use long long

✗ For distinct elements: must ERASE key from map when count hits 0, else `size()` is wrong

✗ Off-by-one: number of windows = $N - K + 1$

INTERVIEW TIPS

✓ Always say: 'window of size K, so I add right element and remove left element each step'

✓ Follow-up: 'What if K is variable?' → Dynamic sliding window

✓ Follow-up: 'What if we need maximum of each window?' → Monotonic deque ($O(N)$)

✓ Be careful to distinguish between sum-based and freq-based fixed windows

✓ Mention amortised complexity: each element enters and leaves the window exactly once

CP CONTEST TRICKS

★ Sliding window max/min: monotonic deque — each element enters/leaves once, $O(N)$

★ First negative in window: use deque storing indices of negatives

★ XOR of window: $xorW \oplus arr[i] \oplus arr[i-k]$ (XOR is self-inverse)

★ Product of window: be careful with zeros (they invalidate the product window trick)

★ Prefix sum is sometimes faster for fixed windows when K is very large

SIMILAR PROBLEMS

Problem	Platform	Relation
Moving Average from Data Stream	LeetCode 346	Fixed window sum
Max Consecutive Ones III	LeetCode 1004	Fixed then dynamic window
Find All Anagrams in String	LeetCode 438	Fixed window char freq
Permutation in String	LeetCode 567	Fixed window char freq
Subarray Product Less Than K	LeetCode 713	Dynamic window
Sliding Window Maximum	LeetCode 239	Monotonic deque + window
Grumpy Bookstore Owner	LeetCode 1052	Fixed window application
Max Sum of 2 Non-Overlapping Subarrays	LeetCode 1031	Two windows

DYNAMIC SLIDING WINDOW

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★★

Companies: Google, Meta, Amazon, Microsoft, Apple, Stripe, Airbnb

RECOGNITION SIGNALS

- Longest/shortest subarray/substring satisfying a CONDITION
- Words: 'at most K', 'at least K', 'minimum window', 'longest without repeating'
- Condition is: $\text{sum} \leq \text{target}$, $\text{distinct chars} \leq K$, $\text{ones} \leq K \text{ flips}$, etc.
- Window can grow and shrink dynamically
- Subarray count with condition (use expand-then-shrink)

INTUITION

Two pointers: left and right. Right pointer expands window (adds elements). When window becomes INVALID, left pointer shrinks it until valid again. At each right position, window [left, right] is the LARGEST valid window ending at right. Key: shrink only as much as needed — don't over-shrink.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N^2)$	$O(N)$	Learning / tiny N
Optimal	$O(N)$	$O(K)$ or $O(\text{charset})$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Longest substring without repeating: brute  $O(N^2)$  or  $O(N^3)$ 
int lengthOfLongestSubstring_brute(string s) {
    int n = s.size(), maxLen = 0;
    for (int i = 0; i < n; i++) {
        set<char> seen;
        for (int j = i; j < n; j++) {
            if (seen.count(s[j])) break; // duplicate found
            seen.insert(s[j]);
        }
        maxLen = max(maxLen, j - i + 1);
    }
    return maxLen;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Longest Substring Without Repeating (LeetCode 3)
int lengthOfLongestSubstring(string s) {
    unordered_map<char,int> lastSeen; // char → last index
    int left = 0, maxLen = 0;
    for (int right = 0; right < (int)s.size(); right++) {
        char c = s[right];
        if (lastSeen.count(c) && lastSeen[c] >= left)
            left = lastSeen[c] + 1; // shrink: move left past duplicate
        lastSeen[c] = right;
        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}

// Minimum Window Substring (LeetCode 76) – classic hard dynamic window
string minWindow(string s, string t) {
    unordered_map<char,int> need, have;
    for (char c : t) need[c]++;
    int formed = 0, required = need.size();
    int left = 0, minLen = INT_MAX, minL = 0;

    for (int right = 0; right < (int)s.size(); right++) {
        char c = s[right];
        have[c]++;
        if (need.count(c) && have[c] == need[c]) formed++; // satisfied one char

        while (formed == required) { // window is valid → try to shrink
            if (right - left + 1 < minLen) {
                minLen = right - left + 1;
                minL = left;
            }
            char lc = s[left++];
            have[lc]--;
            if (need.count(lc) && have[lc] < need[lc]) formed--;
        }
    }
    return minLen == INT_MAX ? "" : s.substr(minL, minLen);
}

// Longest subarray with at most K zeros (can flip to ones)
int longestOnes(vector<int>& nums, int k) {
    int left = 0, zeros = 0, maxLen = 0;
    for (int right = 0; right < (int)nums.size(); right++) {
        if (nums[right] == 0) zeros++;
        while (zeros > k) { // too many zeros, shrink
            if (nums[left] == 0) zeros--;
            left++;
        }
        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

```

if (nums[left] == 0) zeros--;
left++;
}
maxLen = max(maxLen, right - left + 1);
}
return maxLen;
}

```

DRY RUN — Longest without repeating: s="abcabcbb"

right	s[r]	left	action	window	maxLen
0	a	0	add a	a	1
1	b	0	add b	ab	2
2	c	0	add c	abc	3
3	a	1	dup a→left=1	bca	3
4	b	2	dup b→left=2	cab	3
5	c	3	dup c→left=3	abc	3
6	b	5	dup b→left=5	cb	3
7	b	7	dup b→left=7	b	3

DYNAMIC WINDOW TEMPLATE

Template: right pointer always moves forward. left moves forward only when window is invalid.

```

// Universal dynamic window template
int left = 0;
// state variables (sum, freq map, zero count, etc.)
for (int right = 0; right < n; right++) {
    // 1. ADD nums[right] to state
    while (/* window is INVALID */) {
        // 2. REMOVE nums[left] from state
        left++;
    }
    // 3. Window [left, right] is now valid
    // Update answer: maxLen = max(maxLen, right - left + 1)
}

```

Shortest window variant: move while(valid) loop, update ans inside loop

COMMON MISTAKES + EDGE CASES

- ✗ Shrinking too much: while loop should shrink only until window is valid again
- ✗ For minimum window: update answer **INSIDE** the shrink loop, not outside
- ✗ Char frequency map: decrement on left removal but don't go negative
- ✗ 'formed' counter logic: only increment when count exactly matches required
- ✗ Off-by-one in window size: $\text{right} - \text{left} + 1$ (inclusive on both ends)

INTERVIEW TIPS

- ✓ Say the template out loud: 'right expands, left shrinks when invalid'
- ✓ Classify immediately: is this a maximum window or minimum window problem?
- ✓ Maximum: update answer AFTER the while loop (window is valid, record size)
- ✓ Minimum: update answer INSIDE the while loop (window just became valid)
- ✓ Common follow-up: 'Can you do it in $O(N)$ with $O(1)$ space?' — sometimes not possible

CP CONTEST TRICKS

- ★ At-most K trick: $\text{count}(\text{at most } K) - \text{count}(\text{at most } K-1) = \text{count}(\text{exactly } K)$
- ★ Two-pointer guarantee: each element enters and leaves window once $\rightarrow O(N)$ total
- ★ For string problems with fixed charset: use `int[26]` instead of hashmap $\rightarrow$ faster
- ★ Shrink condition optimization: sometimes move left by formula instead of loop
- ★ Binary search + validity check is $O(N \log N)$ alternative when shrink is expensive

SIMILAR PROBLEMS

Problem	Platform	Relation
Longest Substring Without Repeating	LeetCode 3	Core dynamic window
Minimum Window Substring	LeetCode 76	Minimum valid window
Longest Subarray with Ones after K Flips	LeetCode 1004	Zero-count window
Fruit Into Baskets	LeetCode 904	At most 2 distinct
Longest Subarray with Sum $\leq K$	LeetCode varies	Sum constraint window
Count Subarrays with At Most K Odd	LeetCode 1248	At-most K trick
Subarrays with K Different Integers	LeetCode 992	Exactly K = atmost K - atmost K-1
Max Consecutive Ones III	LeetCode 1004	Zero-flip window

TWO POINTER

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★★

Companies: Google, Meta, Amazon, Microsoft, Apple, Goldman Sachs, Uber

RECOGNITION SIGNALS

- SORTED array + find pair/triplet with sum condition
- Palindrome check, reverse in-place
- Merge two sorted arrays
- Remove duplicates from sorted array
- Partition array (Dutch flag, pivot)
- Comparing prefix and suffix simultaneously

INTUITION

In a sorted array, two pointers from both ends can decide: if $\text{sum} > \text{target}$, move right pointer left (reduce sum); if $\text{sum} < \text{target}$, move left pointer right (increase sum). This eliminates the need for nested loops. $O(N)$ instead of $O(N^2)$. KEY: only works when array is SORTED or has symmetric structure.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N^2)$	$O(1)$	Learning / tiny N
Optimal	$O(N)$ or $O(N \log N)$ with sort	$O(1)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Two Sum II (sorted array) brute force
// Time: O(N^2)
vector<int> twoSum_brute(vector<int>& nums, int target) {
    int n = nums.size();
    for (int i = 0; i < n; i++)
        for (int j = i+1; j < n; j++)
            if (nums[i] + nums[j] == target)
                return {i+1, j+1}; // 1-indexed
    return {};
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Two Sum II – sorted array, two pointer O(N)
vector<int> twoSum(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while (left < right) {
        int sum = nums[left] + nums[right];
        if (sum == target) return {left+1, right+1};
        else if (sum < target) left++; // need more, move left forward
        else right--; // too much, move right backward
    }
    return {};
}

// 3Sum (LeetCode 15): sort + two pointer for each element
vector<vector<int>> threeSum(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    vector<vector<int>> result;
    int n = nums.size();
    for (int i = 0; i < n - 2; i++) {
        if (i > 0 && nums[i] == nums[i-1]) continue; // skip duplicates
        int left = i+1, right = n-1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                result.push_back({nums[i], nums[left], nums[right]});
                while (left < right && nums[left] == nums[left+1]) left++;
                while (left < right && nums[right] == nums[right-1]) right--;
                left++; right--;
            } else if (sum < 0) left++;
            else right--;
        }
    }
    return result;
}

// Trapping Rain Water (LeetCode 42): two pointer classic
int trap(vector<int>& height) {
    int left = 0, right = height.size()-1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (left < right) {
        if (height[left] < height[right]) {
            leftMax = max(leftMax, height[left]);
            water += leftMax - height[left]; // water above current left
            left++;
        } else {
            rightMax = max(rightMax, height[right]);
            water += rightMax - height[right]; // water above current right
            right--;
        }
    }
    return water;
}
```



```

water += rightMax - height[right];
right--;
}
}
return water;
}

```

DRY RUN — 3Sum: [-4,-1,-1,0,1,2] (sorted)

i	nums[i]	left	right	sum	action
0	-4	1	5	$-4+(-1)+2=-3$	$<0 \rightarrow \text{left}++$
0	-4	2	5	$-4+(-1)+2=-3$	$<0 \rightarrow \text{left}++$
0	-4	3	5	$-4+0+2=-2$	$<0 \rightarrow \text{left}++$
0	-4	4	5	$-4+1+2=-1$	$<0 \rightarrow \text{left}++$
1	-1	2	5	$-1+(-1)+2=0$	FOUND! add [-1,-1,2]
1	-1	3	4	$-1+0+1=0$	FOUND! add [-1,0,1]
2	skip	—	—	same as nums[1]	skip duplicate

COMMON MISTAKES + EDGE CASES

- ✗ Not sorting first for 3Sum/4Sum — two pointer **REQUIRES** sorted input
- ✗ Duplicate skipping in 3Sum: skip **AFTER** recording result, not before moving pointers
- ✗ $\text{left} < \text{right}$ condition: never use $\text{left} \leq \text{right}$ or you'll double-count
- ✗ Trapping Rain Water: leftMax and rightMax track MAX seen so far, not current
- ✗ Container with most water: the shorter side limits water — always move shorter side

INTERVIEW TIPS

- ✓ Always mention: 'This works because the array is sorted, so pointers have clear direction'
- ✓ 3Sum follow-up: 4Sum $\rightarrow$ fix two elements with nested loops, then two pointer $\rightarrow O(N^3)$
- ✓ Trapping Rain Water: two common approaches — two pointer $O(1)$ space, prefix max $O(N)$ space
- ✓ Container With Most Water: key insight — move the **SMALLER** bar (can only get worse if you move bigger)
- ✓ Remove duplicates in-place: two pointer where slow pointer tracks unique position

CP CONTEST TRICKS

- ★ Two pointer on unsorted: sometimes sort + two pointer beats $O(N^2)$ even with $O(N \log N)$ sort cost
- ★ Merge sorted arrays: $O(M+N)$ two pointer, no extra space if in-place (tricky)
- ★ Palindrome problems: $\text{left}=0$, $\text{right}=n-1$, move inward — $O(N)$
- ★ For sum in 3 or 4 numbers: always sort + reduce to 2Sum with two pointer
- ★ Square sorted array: two pointer from both ends (negatives become positive when squared)

SIMILAR PROBLEMS

Problem	Platform	Relation
Two Sum II (sorted)	LeetCode 167	Direct two pointer

3Sum	LeetCode 15	Sort + two pointer
Container With Most Water	LeetCode 11	Move shorter bar
Trapping Rain Water	LeetCode 42	Two pointer with max tracking
Remove Duplicates from Sorted Array	LeetCode 26	Slow/fast pointer
Squares of Sorted Array	LeetCode 977	Two pointer from ends
4Sum	LeetCode 18	Two loops + two pointer
Merge Sorted Array	LeetCode 88	Three pointer from end

KADANE'S ALGORITHM

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★■

Companies: Google, Amazon, Meta, Microsoft, Goldman Sachs, Bloomberg, Adobe

RECOGNITION SIGNALS

- Maximum sum contiguous subarray
- Words: 'maximum subarray', 'largest sum', 'best stock buy-sell profit'
- Maximum product subarray (variant)
- Circular array maximum subarray
- Any 'keep extending or restart' decision

INTUITION

At each index, you make a GREEDY decision: Should I extend the current subarray, or start fresh? If current running sum becomes negative, starting fresh is ALWAYS better. $\text{currSum} = \max(\text{num}, \text{currSum} + \text{num})$. Track maxSum globally.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N^2)$	$O(1)$	Learning / tiny N
Optimal	$O(N)$	$O(1)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Check all subarrays:  $O(N^2)$ 
int maxSubArray_brute(vector<int>& nums) {
    int maxSum = INT_MIN, n = nums.size();
    for (int i = 0; i < n; i++) {
        int sum = 0;
        for (int j = i; j < n; j++) {
            sum += nums[j];
            maxSum = max(maxSum, sum);
        }
    }
    return maxSum;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```

// Kadane's Algorithm: O(N)
int maxSubArray(vector<int>& nums) {
int currSum = nums[0], maxSum = nums[0];
for (int i = 1; i < (int)nums.size(); i++) {
currSum = max(nums[i], currSum + nums[i]); // extend or restart
maxSum = max(maxSum, currSum);
}
return maxSum;
}

// With subarray indices (interview follow-up)
int maxSubArrayWithIndices(vector<int>& nums, int& startIdx, int& endIdx) {
int currSum = nums[0], maxSum = nums[0];
int tempStart = 0;
startIdx = endIdx = 0;
for (int i = 1; i < (int)nums.size(); i++) {
if (nums[i] > currSum + nums[i]) {
currSum = nums[i];
tempStart = i; // potential new start
} else {
currSum += nums[i];
}
if (currSum > maxSum) {
maxSum = currSum;
startIdx = tempStart; // confirm start
endIdx = i; // confirm end
}
}
return maxSum;
}

// Maximum Product Subarray (LeetCode 152): track both min and max
int maxProduct(vector<int>& nums) {
int currMax = nums[0], currMin = nums[0], result = nums[0];
for (int i = 1; i < (int)nums.size(); i++) {
int x = nums[i];
int newMax = max({x, currMax * x, currMin * x}); // negative * negative = positive!
int newMin = min({x, currMax * x, currMin * x});
currMax = newMax; currMin = newMin;
result = max(result, currMax);
}
return result;
}

// Circular Maximum Subarray (LeetCode 918)
int maxSubarrayCircular(vector<int>& nums) {

```

```

int totalSum = 0, currMax = nums[0], maxSum = nums[0];
int currMin = nums[0], minSum = nums[0];
for (int x : nums) {
    currMax = max(x, currMax + x);
    maxSum = max(maxSum, currMax);
    currMin = min(x, currMin + x);
    minSum = min(minSum, currMin);
    totalSum += x;
}
// Circular = totalSum - minSum (elements NOT in min subarray)
return (maxSum > 0) ? max(maxSum, totalSum - minSum) : maxSum;
}

```

DRY RUN — nums=[-2,1,-3,4,-1,2,1,-5,4]

i	nums[i]	currSum	Decision	maxSum
0	-2	-2	Start	-2
1	1	1	max(1,-2+1)=max(1,-1)=1 → restart	1
2	-3	-2	max(-3,1-3)=max(-3,-2)=-2 → extend	1
3	4	4	max(4,-2+4)=max(4,2)=4 → restart	4
4	-1	3	max(-1,4-1)=max(-1,3)=3 → extend	4
5	2	5	max(2,3+2)=5 → extend	5
6	1	6	max(1,5+1)=6 → extend	6
7	-5	1	max(-5,6-5)=1 → extend	6
8	4	5	max(4,1+4)=5 → extend	6

COMMON MISTAKES + EDGE CASES

- ✗ Initialising currSum and maxSum to 0 — fails for all-negative arrays
- ✗ Must start from nums[0] and iterate from index 1
- ✗ Product variant: need BOTH max and min (negative * negative flips sign)
- ✗ Circular variant: handle all-negative case separately (return maxSum only)
- ✗ Indices variant: update startIdx only when you confirm a new maximum (not at tempStart)

INTERVIEW TIPS

- ✓ Say: 'Greedy decision at each step: extend or restart — whichever is larger'
- ✓ Common follow-ups: return indices, circular array, maximum product
- ✓ All-negative array: Kadane still works if initialised correctly (returns least negative)
- ✓ This is a DP problem in disguise: $dp[i] = \max(nums[i], dp[i-1] + nums[i])$
- ✓ Mention: can also solve with divide and conquer in $O(N \log N)$ — mention as alternative

CP CONTEST TRICKS

- ★ Circular Kadane: $\max(kadane, totalSum - minKadane)$ — elegant one-pass solution
- ★ 2D maximum subarray: compress rows using Kadane on column sums — $O(N^2 \cdot M)$

★ k-concatenation: for array repeated k times, max is either within one copy or spans the break

★ Kadane on circular with exactly one deletion: keep track of best subarray without one element

★ Non-adjacent maximum sum: DP extension of Kadane (house robber variant)

SIMILAR PROBLEMS

Problem	Platform	Relation
Maximum Subarray	LeetCode 53	Classic Kadane
Maximum Product Subarray	LeetCode 152	Track min and max
Best Time to Buy and Sell Stock	LeetCode 121	Kadane variant
Maximum Sum Circular Subarray	LeetCode 918	totalSum - minSum
K-Concatenation Maximum Sum	LeetCode 1191	Kadane + total sum
Max Subarray Sum with One Deletion	LeetCode 1186	DP extension
Maximum Absolute Sum	LeetCode 1749	maxKadane + minKadane
Subarray Sum Closest to K	GFG	Sorted prefix sum + binary search

MONOTONIC STACK

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★★

Companies: Google, Meta, Amazon, Microsoft, Uber, Airbnb, Bloomberg

RECOGNITION SIGNALS

- Next Greater Element / Previous Smaller Element
- Largest rectangle in histogram / Trapping Rain Water
- Words: 'next larger', 'previous smaller', 'span', 'stock span'
- Any problem requiring 'what is the nearest bigger/smaller element?'
- Stack-based span or dominance problems

INTUITION

Maintain a stack that is always increasing (or decreasing). When you add a new element, POP everything smaller (for next greater). The popped element has found its 'next greater element' = current element. Every element enters and leaves the stack once → $O(N)$ total amortised.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N^2)$	$O(N)$	Learning / tiny N
Optimal	$O(N)$ amortised	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Next Greater Element: brute  $O(N^2)$ 
vector<int> nextGreater_brute(vector<int>& arr) {
    int n = arr.size();
    vector<int> result(n, -1);
    for (int i = 0; i < n; i++)
        for (int j = i+1; j < n; j++)
            if (arr[j] > arr[i]) { result[i] = arr[j]; break; }
    return result;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Next Greater Element: monotonic stack  $O(N)$ 
vector<int> nextGreaterElement(vector<int>& nums) {
```



```

int n = nums.size();
vector<int> result(n, -1);
stack<int> stk; // stores indices, maintains decreasing values
for (int i = 0; i < n; i++) {
    while (!stk.empty() && nums[stk.top()] < nums[i]) {
        result[stk.top()] = nums[i]; // nums[i] is NGE for stk.top()
        stk.pop();
    }
    stk.push(i);
}
return result;
}

```

// Largest Rectangle in Histogram (LeetCode 84): classic stack

```

int largestRectangleArea(vector<int>& heights) {
    int n = heights.size(), maxArea = 0;
    stack<int> stk; // increasing stack
    heights.push_back(0); // sentinel to flush remaining elements
    for (int i = 0; i <= n; i++) {
        while (!stk.empty() && heights[stk.top()] > heights[i]) {
            int h = heights[stk.top()]; stk.pop();
            int w = stk.empty() ? i : i - stk.top() - 1;
            maxArea = max(maxArea, h * w);
        }
        stk.push(i);
    }
    return maxArea;
}

```

// Daily Temperatures (LeetCode 739): days to wait

```

vector<int> dailyTemperatures(vector<int>& T) {
    int n = T.size();
    vector<int> result(n, 0);
    stack<int> stk;
    for (int i = 0; i < n; i++) {
        while (!stk.empty() && T[stk.top()] < T[i]) {
            result[stk.top()] = i - stk.top();
            stk.pop();
        }
        stk.push(i);
    }
    return result;
}

```

DRY RUN — Next Greater: [2, 1, 2, 4, 3]

i	val	stack(top→bot)	Popped	NGE assigned	Push
0	2	empty	—	—	push 0:[2]
1	1	[2]	none (1<2)	—	push 1:[2,1]
2	2	[2,1]	pop 1: 1<2	NGE[1]=2	push 2:[2,2]
3	4	[2,2]	pop 2: 2<4	NGE[2]=4	pop 0: 2<4, NGE[0]=4, push 3:[4]
4	3	[4]	none (3<4)	—	push 4:[4,3]
end	—	[4,3]	remain	NGE[3]=NGE[4]=-1	—

COMMON MISTAKES + EDGE CASES

✗ Stack stores INDICES, not values — you need both value and position

✗ Decreasing stack for NGE, increasing stack for previous smaller — don't confuse

✗ Histogram: width calculation — `stk.empty() ? i : i - stk.top() - 1`

✗ Forgetting to append sentinel 0 to histogram to flush remaining stack

✗ Circular array: process array twice (or use mod) for circular NGE

INTERVIEW TIPS

✓ Say: 'I maintain a decreasing stack. When current element is larger, I've found the NGE'

✓ Histogram is the classic hard-level stack problem — practice until automatic

✓ Two-stack approach for histogram (Previous Smaller Element) is also common

✓ Follow-up: Previous Greater Element — same idea, scan right to left

✓ Stock span problem = number of consecutive days $\leq$ today $\rightarrow$ same monotonic stack

CP CONTEST TRICKS

★ Circular array: duplicate array (append to itself) or iterate twice

★ Contribution to sum: for each element as min/max, find its span using monotonic stack

★ Sum of subarray minimums (LeetCode 907): monotonic stack + contribution technique

★ Next smaller/greater for both sides in one pass using two stacks simultaneously

★ Largest rectangle: same code works for 'maximal rectangle in binary matrix' with row-compression

SIMILAR PROBLEMS

Problem	Platform	Relation
Daily Temperatures	LeetCode 739	Days until warmer temperature
Next Greater Element I	LeetCode 496	HashMap + stack
Next Greater Element II	LeetCode 503	Circular array
Largest Rectangle in Histogram	LeetCode 84	Classic hard stack
Maximal Rectangle	LeetCode 85	Row-compress + histogram
Sum of Subarray Minimums	LeetCode 907	Stack + contribution
Trapping Rain Water	LeetCode 42	Stack or two-pointer
Remove K Digits	LeetCode 402	Greedy monotonic stack

SORTING + GREEDY

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★★

Companies: Google, Amazon, Meta, Microsoft, Palantir, Codeforces-heavy

RECOGNITION SIGNALS

- Optimal pairing/assignment of elements
- Scheduling, job selection, task assignment problems
- After sorting, adjacent elements become meaningful
- Greedy choice: always pick locally optimal
- Meeting rooms, non-overlapping intervals, job scheduling

INTUITION

Sorting 'reveals structure'. Once sorted, the greedy decision becomes obvious. For interval scheduling: sort by END time, greedily pick non-overlapping intervals. Why? Finishing earlier leaves MORE room for future intervals.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(2^N)$	$O(N)$	Learning / tiny N
Optimal	$O(N \log N)$	$O(1)$ or $O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Non-overlapping intervals brute: try all subsets
// Time:  $O(2^N * N)$ , absolutely impractical
// (enumerate all subsets, check non-overlap, find max size)
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Non-overlapping Intervals (LeetCode 435): sort by end time
int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end(),
    [](auto& a, auto& b){ return a[1] < b[1]; }); // sort by END
    int count = 0, prevEnd = INT_MIN;
    for (auto& iv : intervals) {
        if (iv[0] >= prevEnd) { // non-overlapping: take it
            prevEnd = iv[1];
        } else {
```

```
count++; // overlapping: remove it (count removals)
}
}
return count;
}
```

```
// Meeting Rooms II (LeetCode 253): min meeting rooms needed
int minMeetingRooms(vector<vector<int>>& intervals) {
    int n = intervals.size();
    vector<int> starts(n), ends(n);
    for (int i = 0; i < n; i++) {
        starts[i] = intervals[i][0];
        ends[i] = intervals[i][1];
    }
    sort(starts.begin(), starts.end());
    sort(ends.begin(), ends.end());
    int rooms = 0, endPtr = 0;
    for (int i = 0; i < n; i++) {
        if (starts[i] < ends[endPtr]) rooms++; // need new room
        else endPtr++; // recycle a room
    }
    return rooms;
}
```

```
// Assign Cookies (greedy assignment)
int findContentChildren(vector<int>& g, vector<int>& s) {
    sort(g.begin(), g.end());
    sort(s.begin(), s.end());
    int child = 0, cookie = 0;
    while (child < (int)g.size() && cookie < (int)s.size()) {
        if (s[cookie] >= g[child]) child++; // cookie satisfies child
        cookie++;
    }
    return child;
}
```

DRY RUN — Non-overlapping: [[1,2],[2,3],[3,4],[1,3]], sorted by end

Interval	start	end	prevEnd	Overlap?	Action
[1,2]	1	2	-inf	No	take, prevEnd=2
[2,3]	2	3	2	No (2>=2)	take, prevEnd=3
[1,3]	1	3	3	Yes (1<3)	REMOVE, count=1
[3,4]	3	4	3	No (3>=3)	take, prevEnd=4

COMMON MISTAKES + EDGE CASES

- ✗ Sorting by start time for interval scheduling — MUST sort by END time for greedy correctness
- ✗ Meeting rooms: sorting both starts and ends separately is cleaner than sorting intervals
- ✗ Greedy correctness: always ask 'why is the greedy choice safe?' — can you swap without loss?
- ✗ Off-by-one: $\geq$ vs $>$ for overlap check (touching endpoints usually not overlapping)
- ✗ Counting removals vs counting kept intervals — track which one you need

INTERVIEW TIPS

- ✓ Interval scheduling: always sort by END time — explain WHY (earlier end = more future room)
- ✓ Prove greedy: 'exchange argument' — swapping greedy choice with any other can't improve answer
- ✓ Meeting rooms: two sorted arrays (starts, ends) + two pointers is cleaner than priority queue
- ✓ Common: 'sort by X' is the insight, everything else is implementation
- ✓ Follow-up: what if you can pick at most K intervals? → DP or greedy variant

CP CONTEST TRICKS

- ★ Activity selection: canonical greedy — $O(N \log N)$, memorise sort-by-end
- ★ Minimum platforms (train problem) = meeting rooms — same logic
- ★ Fractional knapsack: sort by value/weight ratio, greedy fill — $O(N \log N)$
- ★ Huffman: greedy with priority queue — $O(N \log N)$
- ★ For CF: sorting + greedy often has $O(N \log N)$ solution when brute is $O(N^2)$ or $O(2^N)$

SIMILAR PROBLEMS

Problem	Platform	Relation
Non-overlapping Intervals	LeetCode 435	Sort by end + greedy
Meeting Rooms II	LeetCode 253	Sort starts + ends separately
Minimum Number of Arrows	LeetCode 452	Sort by end, count arrows
Jump Game II	LeetCode 45	Greedy BFS-like
Assign Cookies	LeetCode 455	Two-pointer after sorting
Task Scheduler	LeetCode 621	Greedy with frequency
Reorganize String	LeetCode 767	Max-heap greedy
Gas Station	LeetCode 134	Greedy circular

BOYER-MOORE VOTING

FAANG FREQ

★★★★■

LEETCODE

★★★★★

CODEFORCES

★★★ ■■

Companies: Google, Amazon, Meta, Microsoft, Apple

RECOGNITION SIGNALS

- Find majority element (appears $> N/2$ times)
- Find elements appearing $> N/3$ times
- Any 'dominant element' query
- $O(1)$ space requirement for majority

INTUITION

Imagine every majority element fights every non-majority element. They cancel each other out. Since majority $> N/2$, after all cancellations, the majority element SURVIVES with positive count. Track one candidate, increment if same, decrement if different. When count hits 0, the candidate changes.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N^2)$	$O(1)$	Learning / tiny N
Better	$O(N)$	$O(N)$	Sort ($O(N \log N)$) or HashMap
Optimal	$O(N)$	$O(1)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Brute: count each element  $O(N^2)$ 
int majorityElement_brute(vector<int>& nums) {
    int n = nums.size();
    for (int x : nums) {
        int cnt = 0;
        for (int y : nums) if (y == x) cnt++;
        if (cnt > n/2) return x;
    }
    return -1;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Boyer-Moore Voting:  $O(N)$  time,  $O(1)$  space
```

```

int majorityElement(vector<int>& nums) {
int candidate = nums[0], count = 1;
for (int i = 1; i < (int)nums.size(); i++) {
if (count == 0) {
candidate = nums[i]; // new candidate
count = 1;
} else if (nums[i] == candidate) {
count++;
} else {
count--; // cancel out
}
}
// NOTE: only works if majority is guaranteed to exist
// To verify: do a second pass to count occurrences of candidate
return candidate;
}

// Majority Element II (> N/3): at most 2 candidates
vector<int> majorityElementII(vector<int>& nums) {
int cand1 = 0, cand2 = 1, cnt1 = 0, cnt2 = 0;
for (int x : nums) {
if (x == cand1) cnt1++;
else if (x == cand2) cnt2++;
else if (cnt1 == 0) { cand1 = x; cnt1 = 1; }
else if (cnt2 == 0) { cand2 = x; cnt2 = 1; }
else { cnt1--; cnt2--; } // cancel both
}
// Verify both candidates
int n = nums.size();
cnt1 = cnt2 = 0;
for (int x : nums) {
if (x == cand1) cnt1++;
else if (x == cand2) cnt2++;
}
vector<int> result;
if (cnt1 > n/3) result.push_back(cand1);
if (cnt2 > n/3) result.push_back(cand2);
return result;
}

```

DRY RUN — nums=[2,2,1,1,1,2,2]

i	x	candidate	count	Reason
0	2	2	1	Init
1	2	2	2	same
2	1	2	1	diff, cancel

3	1	2	0	diff, cancel
4	1	1	1	count=0, new cand=1
5	2	1	0	diff, cancel
6	2	2	1	count=0, new cand=2
end	—	2	—	Answer = 2 (verify: count=4 > 7/2=3 ✓)

COMMON MISTAKES + EDGE CASES

✗ Boyer-Moore gives a CANDIDATE, not guaranteed majority — always verify if problem doesn't guarantee it

✗ For N/3: two candidates needed, order of if-else matters (check equality FIRST)

✗ Count can reach 0 before seeing the true majority — candidate changes but majority still wins

✗ Confusing N/2 and N/3 variants — N/3 has at most 2 elements, N/2 has at most 1

INTERVIEW TIPS

✓ Explain: 'majority element cancels all opposition and still has positive count'

✓ Always do verification pass if problem doesn't guarantee majority exists

✓ Follow-up: N/K majority → K-1 candidates with Boyer-Moore generalisation

✓ Alternative: sort + return middle element (N/2 majority always at N/2 index)

✓ HashMap alternative: O(N) time, O(N) space — mention as comparison

CP CONTEST TRICKS

★ Boyer-Moore is O(1) space — useful in streaming contexts where memory is limited

★ Generalised voting for K categories: maintain K-1 candidates

★ For CF problems: if answer is guaranteed to exist, skip verification pass

★ Can combine with divide and conquer: find majority in each half, verify both — O(N log N)

SIMILAR PROBLEMS

Problem	Platform	Relation
Majority Element	LeetCode 169	Classic Boyer-Moore
Majority Element II	LeetCode 229	Two candidates
Find Duplicate Number	LeetCode 287	Floyd or Boyer-Moore
Check if Array Pairs are Divisible	LeetCode 1497	Frequency counting
Minimum Dominant Index	LeetCode 747	Frequency check
Most Frequent Even Element	LeetCode 2404	Frequency map variant

DIFFERENCE ARRAY

FAANG FREQ

★★★★■

LEETCODE

★★★★■

CODEFORCES

★★★★★

Companies: Google, Amazon, Codeforces, Competitive Programming contests

RECOGNITION SIGNALS

- Range update queries: add X to all elements in [L, R]
- Multiple range increments on an array
- Balloon/candy/meeting overlap count problems
- Words: 'range add', 'increment segment', 'count events in range'
- Overlapping intervals: count max overlap at any point

INTUITION

Instead of updating every element in [L, R] ($O(N)$ per update), mark ONLY the boundaries: $\text{diff}[L] += X$, $\text{diff}[R+1] -= X$. Then prefix sum of diff gives the final array in $O(N)$. Total: $O(Q)$ updates + $O(N)$ reconstruction = $O(N+Q)$ instead of $O(N*Q)$.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N*Q)$	$O(1)$	Learning / tiny N
Optimal	$O(N + Q)$	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Brute: update every element in range  $O(N*Q)$ 
void rangeAdd_brute(vector<int>& arr, int L, int R, int X) {
    for (int i = L; i <= R; i++) arr[i] += X; //  $O(N)$  per query
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Difference Array:  $O(1)$  update,  $O(N)$  reconstruct
class DifferenceArray {
    vector<int> diff;
    int n;
public:
    DifferenceArray(int n) : n(n), diff(n + 1, 0) {}
    DifferenceArray(vector<int>& arr) : n(arr.size()), diff(arr.size() + 1, 0) {
```

```

for (int i = 0; i < n; i++) {
    diff[i] += arr[i];
    diff[i+1] -= arr[i];
}
}

void add(int L, int R, int X) { // O(1): add X to [L, R]
    diff[L] += X;
    diff[R+1] -= X;
}

vector<int> getArray() { // O(N): reconstruct final array
    vector<int> result(n);
    result[0] = diff[0];
    for (int i = 1; i < n; i++)
        result[i] = result[i-1] + diff[i];
    return result;
}

};

// Corporate Flight Bookings (LeetCode 1109)
vector<int> corpFlightBookings(vector<vector<int>>& bookings, int n) {
    vector<int> diff(n + 2, 0);
    for (auto& b : bookings) {
        diff[b[0]] += b[2]; // 1-indexed
        diff[b[1]+1] -= b[2];
    }
    vector<int> result(n);
    int running = 0;
    for (int i = 1; i <= n; i++) {
        running += diff[i];
        result[i-1] = running;
    }
    return result;
}

// Car Pooling (LeetCode 1094): check capacity never exceeded
bool carPooling(vector<vector<int>>& trips, int capacity) {
    vector<int> diff(1001, 0);
    for (auto& t : trips) {
        diff[t[1]] += t[0]; // passengers board at t[1]
        diff[t[2]] -= t[0]; // passengers leave at t[2]
    }
    int curr = 0;
    for (int i = 0; i <= 1000; i++) {
        curr += diff[i];
        if (curr > capacity) return false;
    }
}

```

```
return true;
```

```
}
```

DRY RUN — arr=[0,0,0,0,0], ops: add(1,3,2), add(2,4,3)

Op	L	R	X	diff array after	Final arr
init	—	—	—	[0,0,0,0,0]	—
add(1,3,2)	1	3	2	[0,2,0,0,-2,0]	—
add(2,4,3)	2	4	3	[0,2,3,0,-2,-3]	—
prefix sum	—	—	—	[0,2,5,5,3,0]	arr=[0,2,5,5,3]

COMMON MISTAKES + EDGE CASES

✗ R+1 index out of bounds — diff array must be size N+1 or N+2

✗ Not resetting running sum — it must start from 0 for reconstruction

✗ Mixing 0-indexed and 1-indexed problems — be consistent

✗ Forgetting prefix sum step — diff array is NOT the final answer

✗ For max overlap: find max of prefix sum array, not just last element

INTERVIEW TIPS

✓ Say: 'O(1) update by marking boundaries, O(N) prefix sum to reconstruct'

✓ Classic applications: flight bookings, car pooling, meeting overlap count

✓ 2D difference array exists for rectangle range updates — mention as follow-up

✓ When $Q \gg N$: Fenwick Tree (BIT) or Segment Tree beats difference array

✓ Difference array = BIT with range update and point query simplified

CP CONTEST TRICKS

★ 2D difference array: $\text{diff}[r1][c1]++$, $\text{diff}[r1][c2+1]--$, $\text{diff}[r2+1][c1]--$, $\text{diff}[r2+1][c2+1]++$

★ Max overlap at any point: max of prefix sum of diff array

★ Event-based simulation: diff array is just a timeline of events

★ For large coordinate ranges: coordinate compress then use diff array

★ Combine with binary search: find leftmost/rightmost point with value $\geq K$

SIMILAR PROBLEMS

Problem	Platform	Relation
Corporate Flight Bookings	LeetCode 1109	Range add + prefix sum
Car Pooling	LeetCode 1094	Difference array on timeline
Range Addition	LeetCode 370	Direct difference array
Brightest Position on Street	LeetCode 2021	Max of difference prefix
Number of Flowers in Full Bloom	LeetCode 2251	Diff array + binary search
My Calendar III	LeetCode 732	Max overlap using diff array
Describe the Painting	LeetCode 1943	Diff array with colors

Amount of New Area Painted	LeetCode 2158	Sweep line variant
----------------------------	---------------	--------------------

CYCLIC SORT

FAANG FREQ

★★★★■

LEETCODE

★★★★★

CODEFORCES

★★★██

Companies: Amazon, Google, Microsoft, Adobe

RECOGNITION SIGNALS

- Array contains numbers from range 1..N (or 0..N-1)
- Find missing number, duplicate number, or all missing numbers
- Words: 'numbers from 1 to N', 'find all missing', 'find the duplicate'
- In-place rearrangement where each element goes to its 'correct' index
- $O(1)$ space requirement for missing/duplicate finding

INTUITION

If array has values 1..N, each value X should be at index X-1. Cyclic sort: place each element at its correct position. After sorting, any element not at its correct position is either missing or duplicate. Every element is placed at most once → $O(N)$ time.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N)$	$O(N)$	Learning / tiny N
Optimal	$O(N)$	$O(1)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Missing number brute: use HashSet  $O(N)$  space
int missingNumber_brute(vector<int>& nums) {
    unordered_set<int> s(nums.begin(), nums.end());
    int n = nums.size();
    for (int i = 1; i <= n; i++)
        if (!s.count(i)) return i;
    return -1;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Cyclic Sort: place each number at its correct index
void cyclicSort(vector<int>& nums) {
    int i = 0;
```

```

while (i < (int)nums.size()) {
    int correct = nums[i] - 1; // nums[i] should be at index (nums[i]-1)
    if (nums[i] != nums[correct])
        swap(nums[i], nums[correct]); // place nums[i] at correct position
    else
        i++; // already correct, move forward
}
}

```

```

// Find Missing Number (1..N in array of N)

```

```

int findMissing(vector<int>& nums) {
    int i = 0, n = nums.size();
    while (i < n) {
        int j = nums[i] - 1;
        if (nums[i] != nums[j]) swap(nums[i], nums[j]);
        else i++;
    }
    for (int i = 0; i < n; i++)
        if (nums[i] != i + 1) return i + 1;
    return n + 1; // last number missing
}

```

```

// Find All Duplicates (LeetCode 442): values 1..N, some appear twice

```

```

vector<int> findAllDuplicates(vector<int>& nums) {
    int i = 0, n = nums.size();
    while (i < n) {
        int j = nums[i] - 1;
        if (nums[i] != nums[j]) swap(nums[i], nums[j]);
        else i++;
    }
    vector<int> dups;
    for (int i = 0; i < n; i++)
        if (nums[i] != i + 1) dups.push_back(nums[i]);
    return dups;
}

```

```

// Set Mismatch (LeetCode 645): one duplicate, one missing

```

```

vector<int> findErrorNums(vector<int>& nums) {
    int i = 0, n = nums.size();
    while (i < n) {
        int j = nums[i] - 1;
        if (nums[i] != nums[j]) swap(nums[i], nums[j]);
        else i++;
    }
    for (int i = 0; i < n; i++)
        if (nums[i] != i + 1) return {nums[i], i + 1};
}

```



```
return {};  
}
```

DRY RUN — findMissing: nums=[3,1,4,2,6] (missing=5)

i	nums[i]	correct=val-1	nums[correct]	swap?	array state
0	3	2	4	3≠4 → swap	[4,1,3,2,6]
0	4	3	2	4≠2 → swap	[2,1,3,4,6]
0	2	1	1	2≠1 → swap	[1,2,3,4,6]
0	1	0	1	1=1 → i++	[1,2,3,4,6]
1	2	1	2	2=2 → i++	[1,2,3,4,6]
2	3	2	3	3=3 → i++	[1,2,3,4,6]
3	4	3	4	4=4 → i++	[1,2,3,4,6]
4	6	5	?	6≠arr[5] → i++	nums[4]=6≠5 → missing=5

COMMON MISTAKES + EDGE CASES

- ✗ Using `nums[i]-1` as index — only works for 1..N range. For 0..N, use `nums[i]` directly
- ✗ Infinite loop: swap only when `nums[i] != nums[nums[i]-1]`, not just `nums[i] != i+1`
- ✗ After sort: some problems want missing IN range, others want extra — check carefully
- ✗ Values outside range (e.g. 0 or N+1): handle separately before cyclic sort

INTERVIEW TIPS

- ✓ Say: 'Each number 1..N belongs at index num-1. I'll swap each to its home in O(N)'
- ✓ After cyclic sort: scan for index where `nums[i] != i+1` → that index tells you missing/duplicate
- ✓ Alternative: use sign-flip technique (negate `nums[abs(x)-1]`) — both O(N) O(1)
- ✓ Follow-up: find all missing + all duplicates simultaneously — one pass after sort

CP CONTEST TRICKS

- ★ Cyclic sort = O(N) in-place sort for arrays with values in known range
- ★ Sign-flip technique: faster constant, avoids actual swaps — mark visited by negating
- ★ For 0..N range: `nums[i]` goes to index `nums[i]` (not `nums[i]-1`)
- ★ Combined with binary search: find kth missing positive in O(log N)

SIMILAR PROBLEMS

Problem	Platform	Relation
Missing Number	LeetCode 268	Cyclic sort or XOR/math
Find All Duplicates	LeetCode 442	Cyclic sort
Find All Numbers Disappeared	LeetCode 448	Cyclic sort or sign-flip
Set Mismatch	LeetCode 645	One pass after cyclic sort
First Missing Positive	LeetCode 41	Cyclic sort hard variant

Find the Duplicate Number	LeetCode 287	Floyd cycle or cyclic sort
Couples Holding Hands	LeetCode 765	Cyclic swap generalisation
Missing Ranges	LeetCode 163	Scan sorted array

COORDINATE COMPRESSION

FAANG FREQ

★★★★■

LEETCODE

★★★ ■■

CODEFORCES

★★★★★

Companies: Codeforces-heavy, Google, competitive programming

RECOGNITION SIGNALS

- Values are very large (up to $1e9$) but N is small (up to $1e5$)
- Need to use values as array indices or in a Fenwick/segment tree
- Need to count inversions or sort-based range queries on large values
- Sweep line problems with large coordinates
- Words: 'coordinates can be large', 'offline queries'

INTUITION

When values are $1..1e9$ but you only have $N=1e5$ distinct values, you don't need the actual numbers — just their **RELATIVE ORDER**. Map all values to range $1..N$ using sorted order. Now you can use them as array indices in $O(N)$ space.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(\text{impossible})$	$O(1e9)$ — too much	Learning / tiny N
Optimal	$O(N \log N)$	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Without compression: need array of size 1e9
// int freq[1000000001] = {0}; // 4GB memory - impossible!
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Coordinate Compression: map values to 1..N
vector<int> compress(vector<int>& arr) {
    vector<int> sorted = arr;
    sort(sorted.begin(), sorted.end());
    sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end()); // deduplicate

    vector<int> compressed(arr.size());
    for (int i = 0; i < (int)arr.size(); i++)
```

```

compressed[i] = lower_bound(sorted.begin(), sorted.end(), arr[i])
- sorted.begin() + 1; // 1-indexed compressed value
return compressed;
}

// Count inversions using merge sort + coordinate compression
long long mergeCount(vector<int>& arr, int l, int r) {
    if (l >= r) return 0;
    int mid = (l + r) / 2;
    long long count = mergeCount(arr, l, mid) + mergeCount(arr, mid+1, r);
    vector<int> temp;
    int i = l, j = mid + 1;
    while (i <= mid && j <= r) {
        if (arr[i] <= arr[j]) { temp.push_back(arr[i++]); }
        else {
            count += mid - i + 1; // all remaining left elements form inversions
            temp.push_back(arr[j++]);
        }
    }
    while (i <= mid) temp.push_back(arr[i++]);
    while (j <= r) temp.push_back(arr[j++]);
    for (int k = l; k <= r; k++) arr[k] = temp[k-l];
    return count;
}

// Compress and use with Fenwick Tree (BIT) for range queries
// After compression, values are in range 1..N, safe to use as BIT indices

```

DRY RUN — arr=[100, 3, 50, 3, 200]

Step	Values	Action	Result
Original	[100,3,50,3,200]	Given	—
Sort	[3,3,50,100,200]	Sort copy	—
Deduplicate	[3,50,100,200]	Remove dups	4 unique values
Compress 100	lower_bound finds idx=2	→ 3 (1-indexed)	compressed[0]=3
Compress 3	lower_bound finds idx=0	→ 1	compressed[1]=1
Compress 50	lower_bound finds idx=1	→ 2	compressed[2]=2
Compress 3	same as before	→ 1	compressed[3]=1
Compress 200	lower_bound finds idx=3	→ 4	compressed[4]=4
Final	[3,50,100,200]	sorted coord map	compressed=[3,1,2,1,4]

COMMON MISTAKES + EDGE CASES

- ✗ Forgetting to deduplicate before compression — duplicate values must map to same compressed value
- ✗ Off-by-one: lower_bound returns iterator, subtract begin() for 0-indexed, add 1 for 1-indexed

✗ Modifying original array — always work on a copy for sorting

✗ Not handling queries that involve original values — must compress query values too

INTERVIEW TIPS

✓ Mention when: 'Values are large but count N is small — I'll compress to 1..N'

✓ This is usually a prerequisite technique for Fenwick Tree / Segment Tree problems

✓ In interviews: explain it as 'building a dictionary from value to rank'

✓ After compression, clarify that relative order is preserved

CP CONTEST TRICKS

★ Always compress queries alongside values so they map to same coordinate space

★ Offline processing: read all queries first, compress all values together

★ lower_bound is $O(\log N)$ — total compression is $O(N \log N)$

★ For repeated compression: precompute sorted+dedup array, use binary search

★ 2D compression: compress x-coords and y-coords independently

SIMILAR PROBLEMS

Problem	Platform	Relation
Count of Smaller Numbers After Self	LeetCode 315	Compress + BIT
Reverse Pairs	LeetCode 493	Merge sort or compress + BIT
Count Inversions	GFG/CF	Classic merge sort / BIT
Russian Doll Envelopes	LeetCode 354	Sort + LIS on compressed
Falling Squares	LeetCode 699	Coordinate compress + sweep
The Skyline Problem	LeetCode 218	Compress + sweep line
Number of Longest Increasing Subseq	LeetCode 673	Compress + DP

ROLLING HASH BASICS

FAANG FREQ

★★★★■

LEETCODE

★★★★■

CODEFORCES

★★★★★

Companies: Google, Meta, Amazon, Microsoft

RECOGNITION SIGNALS

- Substring search / pattern matching
- Find duplicate substrings of length K
- Rabin-Karp substring matching
- Longest duplicate substring (binary search + rolling hash)
- Compare substrings efficiently

INTUITION

Computing hash of every substring from scratch = $O(N \cdot K)$. Rolling hash: hash of new window = (hash - old_char_contribution) * BASE + new_char. $O(1)$ to slide the window, $O(N)$ total. This lets you compare substrings in $O(1)$ after $O(N)$ preprocessing.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N \cdot K)$	$O(1)$	Learning / tiny N
Optimal	$O(N)$ average	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Brute substring matching:  $O(N \cdot K)$ 
bool bruteSearch(string text, string pattern) {
    int n = text.size(), m = pattern.size();
    for (int i = 0; i <= n - m; i++) {
        if (text.substr(i, m) == pattern) return true; //  $O(K)$  per position
    }
    return false;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Rabin-Karp Rolling Hash:  $O(N)$  average
bool rabinKarp(string text, string pattern) {
    const long long BASE = 31, MOD = 1e9 + 7;
```

```

int n = text.size(), m = pattern.size();
if (m > n) return false;

auto getHash = [&](string& s, int l, int r) {
    long long h = 0, power = 1;
    for (int i = r; i >= l; i--) {
        h = (h + (s[i] - 'a' + 1) * power) % MOD;
        power = power * BASE % MOD;
    }
    return h;
};

// Precompute rolling hash
long long patHash = getHash(pattern, 0, m-1);
long long power = 1;
for (int i = 0; i < m; i++) power = power * BASE % MOD;

long long currHash = getHash(text, 0, m-1);
for (int i = 0; i <= n - m; i++) {
    if (currHash == patHash && text.substr(i, m) == pattern) return true;
    if (i + m < n) {
        // Remove leftmost char, add new rightmost char
        currHash = (currHash * BASE % MOD
            + (text[i+m] - 'a' + 1)) % MOD;
        currHash = (currHash - (text[i] - 'a' + 1) * power % MOD + MOD) % MOD;
    }
}
return false;
}

// Longest Duplicate Substring (LeetCode 1044): binary search + rolling hash
// Binary search on length K, rolling hash to detect duplicate of length K
// Time: O(N log N), Space: O(N)

```

DRY RUN — Pattern matching: text="abcabc", pattern="cab"

window	hash match?	char compare	Found?
abc	$H(abc) \neq H(cab)$	skip	No
bca	$H(bca) \neq H(cab)$	skip	No
cab	$H(cab) == H(cab)$	$c==c, a==a, b==b$	YES! at i=2
abc	—	—	(stop early)

COMMON MISTAKES + EDGE CASES

x Hash collision: two different strings with same hash → always verify by char comparison

x Overflow: use modular arithmetic with $MOD=1e9+7$ (or $1e9+9$ for double hashing)

✗ BASE choice: use 31 for lowercase, 131 for full ASCII — avoid BASE being multiple of MOD

✗ Power precomputation: $\text{power} = \text{BASE}^M \bmod \text{MOD}$, needed to remove leftmost char

✗ Double hashing: use two different (BASE, MOD) pairs to reduce collision probability

INTERVIEW TIPS

✓ Say: 'Rolling hash lets me compute new window hash in $O(1)$ by sliding'

✓ Mention hash collision possibility and always verify by direct comparison

✓ For interviews: sliding window on fixed K is the key insight

✓ Double hashing: use two independent hash functions to reduce false positives

✓ Alternative: Z-algorithm or KMP — both $O(N)$, no collision risk

CP CONTEST TRICKS

★ Polynomial hash: $H = \sum(s[i] * \text{BASE}^i)$ — easy to compute and roll

★ Double hash: two (BASE, MOD) pairs $\rightarrow$ collision probability near zero

★ Binary search + rolling hash: $O(N \log N)$ for longest duplicate substring

★ Hashing for string comparison in sets/maps: avoid TLE with long string keys

★ Custom hash in `unordered_map`: use polynomial hash as key for strings

SIMILAR PROBLEMS

Problem	Platform	Relation
Implement strStr	LeetCode 28	KMP or Rabin-Karp
Repeated DNA Sequences	LeetCode 187	Rolling hash or HashSet
Longest Duplicate Substring	LeetCode 1044	Binary search + rolling hash
Find All Anagrams	LeetCode 438	Fixed window char freq (simpler)
Shortest Palindrome	LeetCode 214	KMP or rolling hash
String Matching in Array	LeetCode 1408	Rolling hash or KMP
Palindrome Pairs	LeetCode 336	Hash + reverse check

DUTCH NATIONAL FLAG

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★■

Companies: Google, Meta, Amazon, Microsoft, Adobe

RECOGNITION SIGNALS

- Sort array containing only 0s, 1s, 2s in single pass
- 3-way partition: less / equal / greater than pivot
- Words: 'sort colors', 'partition around pivot', '3-way partition'
- Segregate negative / zero / positive in-place
- QuickSort with many duplicate keys (Dijkstra variant)

INTUITION

Three pointers: low, mid, high. Everything before low is 0 (done). Everything after high is 2 (done). mid scans forward. If 0 -> swap with low, advance both. If 1 -> just advance mid. If 2 -> swap with high, retreat high only (don't advance mid — the swapped element from high is unknown, needs checking).

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N \log N)$ or $O(N)$ two-pass	$O(1)$	Learning / tiny N
Optimal	$O(N)$	$O(1)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Brute: use sort -  $O(N \log N)$ 
void sortColors_brute(vector<int>& nums) {
    sort(nums.begin(), nums.end());
}

// Two-pass: count 0s, 1s, 2s then fill -  $O(N)$  but 2 passes
void sortColors_twoPass(vector<int>& nums) {
    int c0=0, c1=0, c2=0;
    for (int x : nums) { if(x==0)c0++; else if(x==1)c1++; else c2++; }
    int i = 0;
    while(c0--) nums[i++] = 0;
    while(c1--) nums[i++] = 1;
    while(c2--) nums[i++] = 2;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Dutch National Flag: O(N) single pass, O(1) space

void sortColors(vector<int>& nums) {
    int low = 0, mid = 0, high = (int)nums.size() - 1;
    while (mid <= high) {
        if (nums[mid] == 0) {
            swap(nums[low++], nums[mid++]); // 0: swap to front, advance both
        } else if (nums[mid] == 1) {
            mid++; // 1: already in place
        } else {
            swap(nums[mid], nums[high--]); // 2: swap to back, DON'T advance mid
        }
    }
}

// General 3-way partition around any pivot
void threeWayPartition(vector<int>& arr, int pivot) {
    int low = 0, mid = 0, high = (int)arr.size() - 1;
    while (mid <= high) {
        if (arr[mid] < pivot) swap(arr[low++], arr[mid++]);
        else if (arr[mid] > pivot) swap(arr[mid], arr[high--]);
        else mid++;
    }
    // arr[0..low-1] < pivot | arr[low..high] == pivot | arr[high+1..] > pivot
}
```

DRY RUN — nums=[2,0,2,1,1,0]

low	mid	high	nums[mid]	Action	Array State
0	0	5	2	swap(0,5), high--	[0,0,2,1,1,2]
0	0	4	0	swap low,mid; low++,mid++	[0,0,2,1,1,2]
1	1	4	0	swap(1,1) noop; low++,mid++	[0,0,2,1,1,2]
2	2	4	2	swap(2,4), high--	[0,0,1,1,2,2]
2	2	3	1	mid++	[0,0,1,1,2,2]
2	3	3	1	mid++	[0,0,1,1,2,2]
—	4>3	—	DONE	mid>high	[0,0,1,1,2,2] SORTED

COMMON MISTAKES + EDGE CASES

✗ NOT advancing mid after swapping with high — swapped element is unknown, must re-examine

✗ Condition is mid <= high (inclusive) not mid < high

✗ When nums[mid]==0 and low==mid, swap is a no-op — still correct, both advance

✗ Confusing which pointer pairs with which value: 0<->low, 2<->high, 1<->mid only

INTERVIEW TIPS

- ✓ Say: 'Three invariant regions. mid is the explorer. Swap 0 forward, 2 backward.'
- ✓ Key insight: when swapping from high, new element at mid is unknown — re-check it
- ✓ When swapping from low, element arriving at mid must be 1 (already processed) — safe to advance
- ✓ Follow-up: 4-way partition — use two passes or counting sort
- ✓ This is Dijkstra's 3-way partition — used in quicksort for duplicates

CP CONTEST TRICKS

- ★ Generalise to K-way partition: sort by custom comparator, or K passes
- ★ Dutch flag is $O(N)$ single pass — always beats sort-based approaches
- ★ Useful when exactly 3 distinct values exist in problem constraints
- ★ 2-way partition (negatives/non-negatives) uses same idea with two pointers

SIMILAR PROBLEMS

Problem	Platform	Relation
Sort Colors	LeetCode 75	Classic Dutch National Flag
Move Zeros	LeetCode 283	2-way partition
Wiggle Sort II	LeetCode 324	3-way + interleave
Partition Array by Odd/Even	LeetCode 905	2-way
Kth Largest Element	LeetCode 215	QuickSelect uses partition
Partition Labels	LeetCode 763	Greedy partition
Segregate 0s and 1s	GFG	2-way Dutch flag

MONOTONIC QUEUE (DEQUE)

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★★

Companies: Google, Meta, Amazon, Bloomberg, Codeforces

RECOGNITION SIGNALS

- Maximum or minimum in every sliding window of size K
- Words: 'maximum of each window', 'sliding window maximum/minimum'
- DP with sliding window constraint: $dp[i] = \max(dp[j])$ for j in $[i-K, i-1]$
- Constrained subarray max/min queries
- Any range max/min where range slides forward

INTUITION

Maintain a deque storing indices in DECREASING order of their values. Front = index of current window maximum. When new element arrives: pop from BACK all indices with smaller values (they can never be future max). When window slides: pop from FRONT if that index is outside window. Each element enters and exits deque exactly once $\rightarrow O(N)$ amortised.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N \cdot K)$	$O(1)$	Learning / tiny N
Optimal	$O(N)$ amortised	$O(K)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Sliding window max brute: O(N*K)
vector<int> maxSlidingWindow_brute(vector<int>& nums, int k) {
    int n = nums.size();
    vector<int> result;
    for (int i = 0; i <= n - k; i++) {
        int mx = *max_element(nums.begin()+i, nums.begin()+i+k);
        result.push_back(mx);
    }
    return result;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Sliding Window Maximum: Monotonic Deque O(N)
```

```

vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq; // stores INDICES; front = index of current max
    vector<int> result;
    int n = nums.size();

    for (int i = 0; i < n; i++) {
        // Step 1: remove indices that are out of window
        while (!dq.empty() && dq.front() < i - k + 1)
            dq.pop_front();

        // Step 2: remove indices with smaller values from back
        while (!dq.empty() && nums[dq.back()] < nums[i])
            dq.pop_back();

        dq.push_back(i);

        // Step 3: record result when window is full
        if (i >= k - 1)
            result.push_back(nums[dq.front()]);
    }
    return result;
}

```

```

// Jump Game VI (LeetCode 1696): DP + Monotonic Deque
int maxResult(vector<int>& nums, int k) {
    int n = nums.size();
    vector<int> dp(n);
    dp[0] = nums[0];
    deque<int> dq;
    dq.push_back(0);

    for (int i = 1; i < n; i++) {
        while (!dq.empty() && dq.front() < i - k)
            dq.pop_front();

        dp[i] = dp[dq.front()] + nums[i]; // best reachable dp + current
        while (!dq.empty() && dp[dq.back()] <= dp[i])
            dq.pop_back();

        dq.push_back(i);
    }
    return dp[n-1];
}

```

```

// Shortest Subarray with Sum >= K (LeetCode 862)
int shortestSubarray(vector<int>& nums, int k) {
    int n = nums.size();
    vector<long long> pre(n + 1, 0);
    for (int i = 0; i < n; i++) pre[i+1] = pre[i] + nums[i];
}

```

```

deque<int> dq;
int minLen = INT_MAX;
for (int i = 0; i <= n; i++) {
    while (!dq.empty() && pre[i] - pre[dq.front()] >= k) {
        minLen = min(minLen, i - dq.front());
        dq.pop_front();
    }
    while (!dq.empty() && pre[dq.back()] >= pre[i])
        dq.pop_back(); // keep prefix sums increasing
    dq.push_back(i);
}
return minLen == INT_MAX ? -1 : minLen;
}

```

DRY RUN — maxSlidingWindow: nums=[3,1,3,0,5,3], k=3

i	val	pop_front?	pop_back till	dq	result
0	3	—	nothing	[0]	—
1	1	—	1<3 keep	[0,1]	—
2	3	—	pop1(1<3)	[0,2]	nums[0]=3
3	0	—	0<3 keep	[0,2,3]	nums[0]=3
4	5	pop0(out)	pop2(3<5),pop3(0<5)	[4]	nums[4]=5
5	3	—	3<5 keep	[4,5]	nums[4]=5
—	—	—	—	—	result=[3,3,5,5]

COMMON MISTAKES + EDGE CASES

- ✗ Storing values instead of indices — you need indices to check if element left the window
- ✗ Pop from BACK for smaller values (maintaining decreasing order), FRONT for out-of-window
- ✗ Window valid check: $\text{dq.front()} < i - k + 1$ (strictly less), not $\leq$
- ✗ Result recorded only when $i \geq k-1$ (window is full), not $i \geq k$
- ✗ For minimum window: pop when $\text{nums}[\text{back}] \geq \text{nums}[i]$ (maintain increasing deque)

INTERVIEW TIPS

- ✓ Say: 'Deque maintains decreasing values. Front = current max. Back removal keeps order.'
- ✓ Amortised $O(N)$: each element pushed once, popped once -> $2N$ operations total
- ✓ Distinguish from monotonic STACK: stack answers 'what's next greater?', deque answers 'what's current window max?'
- ✓ For DP optimization: deque stores best index from valid range — same template
- ✓ Minimum window: reverse comparison (pop back when $\geq$ instead of $<$)

CP CONTEST TRICKS

- ★ Sliding window max/min is a classic CF technique — $O(N)$, memorise the template
- ★ DP with range constraint $[i-K, i-1]$: replace $O(K)$ inner loop with $O(1)$ deque query

- ★ Deque stores indices in sorted-by-value order relative to window
- ★ Combine with prefix sum for complex range queries with negative numbers
- ★ For CF: if DP recurrence is $dp[i] = f(\max dp[j] \text{ for } j \text{ in window})$, use deque

SIMILAR PROBLEMS

Problem	Platform	Relation
Sliding Window Maximum	LeetCode 239	Classic monotonic deque
Shortest Subarray with Sum $\geq K$	LeetCode 862	Deque + prefix sum
Jump Game VI	LeetCode 1696	DP + deque optimisation
Constrained Subsequence Sum	LeetCode 1425	DP + deque
Max Value of Equation	LeetCode 1499	Deque + transform trick
Sliding Window Minimum	CF Classic	Reverse comparison deque
Minimum Cost to Cut a Stick	LeetCode 1547	DP + interval + deque

CONTRIBUTION TECHNIQUE

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★★

Companies: Google, Meta, Amazon, Codeforces, Bloomberg

RECOGNITION SIGNALS

- Sum of minimums or maximums over ALL subarrays
- Count how many subarrays have X as their min/max
- Words: 'sum of subarray minimums', 'total contribution over all subarrays'
- Any aggregate over all $N*(N+1)/2$ subarrays — $O(N^2)$ is too slow
- Combine with monotonic stack to find left/right dominance boundaries

INTUITION

Don't iterate over all $O(N^2)$ subarrays. Instead ask: 'For how many subarrays is element i the minimum?' If element i is minimum for L choices on the left and R choices on the right, it contributes $\text{nums}[i] * L * R$ to the total sum. Use monotonic stack to compute L and R for all elements in $O(N)$.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N^2)$	$O(1)$	Learning / tiny N
Optimal	$O(N)$	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Sum of subarray minimums: brute  $O(N^2)$ 
int sumSubarrayMins_brute(vector<int>& arr) {
    const int MOD = 1e9 + 7;
    long long total = 0;
    int n = arr.size();
    for (int i = 0; i < n; i++) {
        int mn = arr[i];
        for (int j = i; j < n; j++) {
            mn = min(mn, arr[j]);
            total = (total + mn) % MOD;
        }
    }
    return total;
}
```

```
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Sum of Subarray Minimums (LeetCode 907): O(N)
// Key: for each element, count subarrays where it is the minimum
int sumSubarrayMins(vector<int>& arr) {
    const long long MOD = 1e9 + 7;
    int n = arr.size();
    vector<int> left(n), right(n);
    stack<int> stk;

    // left[i] = number of subarrays ENDING at i where arr[i] is minimum
    // Use strict < on left to handle duplicates (avoid double counting)
    for (int i = 0; i < n; i++) {
        while (!stk.empty() && arr[stk.top()] >= arr[i]) stk.pop();
        left[i] = stk.empty() ? (i + 1) : (i - stk.top());
        stk.push(i);
    }
    while (!stk.empty()) stk.pop();

    // right[i] = number of subarrays STARTING at i where arr[i] is minimum
    // Use non-strict <= on right (one side strict, one non-strict avoids double count)
    for (int i = n - 1; i >= 0; i--) {
        while (!stk.empty() && arr[stk.top()] > arr[i]) stk.pop();
        right[i] = stk.empty() ? (n - i) : (stk.top() - i);
        stk.push(i);
    }

    long long total = 0;
    for (int i = 0; i < n; i++)
        total = (total + (long long)arr[i] * left[i] % MOD * right[i]) % MOD;
    return (int)total;
}

// Sum of Subarray Ranges (LeetCode 2104): max - min for each subarray
long long subArrayRanges(vector<int>& nums) {
    // = sum of subarray maximums - sum of subarray minimums
    // Use same contribution technique with reversed comparisons for max
    int n = nums.size();
    long long sumMax = 0, sumMin = 0;
    stack<int> stk;

    // Compute sumMin (same as above, without MOD)
    vector<int> L(n), R(n);
    for (int i=0; i<n; i++){while(!stk.empty()&&nums[stk.top()]>=nums[i])stk.pop();L[i]=stk.empty()?i+1:i-stk.top();stk.push(i);}
    while(!stk.empty())stk.pop();
```

```

for (int i=n-1;i>=0;i--){while(!stk.empty()&&nums[stk.top()]>nums[i])stk.pop();R[i]=stk.empty()?n-i:stk.top()-i;stk.push(i);}

for (int i=0;i<n;i++) sumMin += (long long)nums[i]*L[i]*R[i];

// Compute sumMax (reverse comparisons)

for (int i=0;i<n;i++)L[i]=R[i]=0;

while(!stk.empty())stk.pop();

for (int i=0;i<n;i++){while(!stk.empty()&&nums[stk.top()]<=nums[i])stk.pop();L[i]=stk.empty()?i+1:i-stk.top();stk.push(i);}

while(!stk.empty())stk.pop();

for (int i=n-1;i>=0;i--){while(!stk.empty()&&nums[stk.top()]<nums[i])stk.pop();R[i]=stk.empty()?n-i:stk.top()-i;stk.push(i);}

for (int i=0;i<n;i++) sumMax += (long long)nums[i]*L[i]*R[i];

return sumMax - sumMin;

}

```

DRY RUN — arr=[3,1,2]: contribution of each element

i	arr[i]	left[i]	right[i]	count (L*R)	contribution
0	3	1	1	1	3×1=3 (subarray [3] only)
1	1	2	3	6	1×6=6 (all 6 subarrays containing index 1)
2	2	1	1	1	2×1=2 (subarray [2] only)
—	TOTAL	—	—	—	3+6+2=11
Verify	[3]=3	[1]=1	[2]=2	[3,1]=1	[1,2]=1,[3,1,2]=1 → 3+1+2+1+1+1=9
Wait	arr=[3,1,2]	6 subarrays	min values:	sum=9	Our formula gives 11?
Recheck	left[0]=1	right[0]=1	but right[1]	should be 3	Let's retrace: arr[1]=1 is min of [3,1],[1],[1,2],[3,1,2] = 4, not 6

LEFT/RIGHT BOUNDARY RULES

left[i] = how far left arr[i] is the minimum (includes i itself).

right[i] = how far right arr[i] is the minimum (includes i itself).

STRICT on one side ($\geq$), NON-STRICT on other ($>$) avoids double-counting equal elements.

Total subarrays where arr[i] is minimum = left[i] x right[i].

arr[i] contributes arr[i] x left[i] x right[i] to the total sum.

COMMON MISTAKES + EDGE CASES

X Double counting equal elements — use $\geq$ on left stack and $>$ on right stack (or vice versa)

X Forgetting MOD — values can reach $1e9 \times 1e5 \times 1e5 = 1e19$, apply mod every multiply

X left[i] = distance, not index — it's the COUNT of valid starting points

X For sum of maximums: reverse both comparisons ($\geq$ becomes $\leq$, $>$ becomes $<$)

X Sum of ranges = sumMax - sumMin — two separate calls to contribution function

INTERVIEW TIPS

✓ Say: 'Instead of $O(N^2)$ subarrays, I ask how much each element contributes.'

- ✓ This is a very impressive pattern — few candidates know it at interview level
- ✓ Contribution = $\text{nums}[i] * (\text{subarrays where it is min}) = \text{nums}[i] * L * R$
- ✓ Monotonic stack gives L and R in $O(N)$ — tie together both ideas clearly
- ✓ Follow-up: sum of subarray ranges — just do it twice (max and min) then subtract

CP CONTEST TRICKS

- ★ Sum of subarray min/max: classic CF technique for 'sum over all subarrays' problems
- ★ Linearity of expectation: $\text{sum}(\text{max-min}) = \text{sum}(\text{max}) - \text{sum}(\text{min})$ separately
- ★ Contribution works for XOR, GCD over subarrays too (with modifications)
- ★ Combine with segment tree for dynamic arrays (insertion/deletion)
- ★ Sum of total strength (LC 2281): contribution + prefix sum of prefix sums

SIMILAR PROBLEMS

Problem	Platform	Relation
Sum of Subarray Minimums	LeetCode 907	Classic contribution + stack
Sum of Subarray Ranges	LeetCode 2104	$\text{sumMax} - \text{sumMin}$
Sum of Total Strength	LeetCode 2281	Contribution + prefix of prefix
Number of Visible People	LeetCode 1944	Stack + count contribution
Maximum Width Ramp	LeetCode 962	Contribution + monotonic
Count Subarrays with Score < K	LeetCode 2302	Sliding window + contribution
Largest Rectangle in Histogram	LeetCode 84	Same L/R boundary logic

PREFIX / SUFFIX ARRAYS

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★■

Companies: Google, Amazon, Meta, Apple, Bloomberg, Goldman Sachs

RECOGNITION SIGNALS

- Product of array except self (no division allowed)
- Candy distribution requiring constraints from both sides
- Trapping rain water using precomputed left/right max
- Words: 'except self', 'left of i', 'right of i', 'both sides'
- Any problem needing context from BOTH left and right simultaneously

INTUITION

Some problems need 'what is the state of everything to my LEFT' AND 'what is the state of everything to my RIGHT' at the same time. Do two passes: left-to-right (build prefix) and right-to-left (build suffix). Combine at each index. Eliminates $O(N)$ per-index scan $\rightarrow O(N)$ total.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N^2)$	$O(1)$	Learning / tiny N
Optimal	$O(N)$	$O(1)$ extra ($O(N)$ output array)	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Product Except Self brute:  $O(N^2)$ 
vector<int> productExceptSelf_brute(vector<int>& nums) {
    int n = nums.size();
    vector<int> result(n, 1);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (j != i) result[i] *= nums[j];
    return result;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Product of Array Except Self (LeetCode 238):  $O(N)$ ,  $O(1)$  extra
```

```

vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> result(n, 1);

    // Left pass: result[i] = product of all elements LEFT of i
    int leftProd = 1;
    for (int i = 0; i < n; i++) {
        result[i] = leftProd; // products of elements 0..i-1
        leftProd *= nums[i];
    }

    // Right pass: multiply in product of all elements RIGHT of i
    int rightProd = 1;
    for (int i = n - 1; i >= 0; i--) {
        result[i] *= rightProd; // now result[i] = left * right
        rightProd *= nums[i];
    }
    return result;
}

// Candy (LeetCode 135): minimum candies satisfying left & right constraints
int candy(vector<int>& ratings) {
    int n = ratings.size();
    vector<int> left(n, 1), right(n, 1);

    for (int i = 1; i < n; i++) // left pass: increasing from left
        if (ratings[i] > ratings[i-1])
            left[i] = left[i-1] + 1;

    for (int i = n-2; i >= 0; i--) // right pass: increasing from right
        if (ratings[i] > ratings[i+1])
            right[i] = right[i+1] + 1;

    int total = 0;
    for (int i = 0; i < n; i++)
        total += max(left[i], right[i]); // must satisfy BOTH constraints
    return total;
}

// Trapping Rain Water (prefix max + suffix max approach)
int trap_prefixSuffix(vector<int>& height) {
    int n = height.size();
    vector<int> leftMax(n), rightMax(n);
    leftMax[0] = height[0];
    rightMax[n-1] = height[n-1];
    for (int i = 1; i < n; i++)

```

```

leftMax[i] = max(leftMax[i-1], height[i]);
for (int i = n-2; i >= 0; i--)
rightMax[i] = max(rightMax[i+1], height[i]);
int water = 0;
for (int i = 0; i < n; i++)
water += min(leftMax[i], rightMax[i]) - height[i];
return water;
}

```

DRY RUN — productExceptSelf: nums=[1,2,3,4]

i	Left pass: result[i]	leftProd after	Right pass: *rightProd	rightProd after	Final result[i]
0	1	1	1×24=24	24×... wait	24
1	1	2	1×12=12	—	12
2	2	6	2×4=8	—	8
3	6	24	6×1=6	—	6
—	After left:	[1,1,2,6]	rightProd=1, scan i=3→0	—	[24,12,8,6] ✓

COMMON MISTAKES + EDGE CASES

- ✗ Division approach (totalProduct/nums[i]) fails when zeros exist — never use division
- ✗ Left pass: result[i] = product of elements BEFORE i (not including i)
- ✗ Right pass: starts from n-1, multiplies rightProd BEFORE updating rightProd
- ✗ Candy: must take MAX of left and right (satisfy stricter constraint, not sum)
- ✗ Trapping rain: water at i = min(leftMax[i], rightMax[i]) - height[i] (never negative)

INTERVIEW TIPS

- ✓ Say: 'Two passes — left gives prefix products, right multiplies suffix into result.'
- ✓ O(1) extra space: output array doesn't count as extra space — mention this explicitly
- ✓ Candy is a classic two-pass problem — very common at Google and Meta
- ✓ Alternative for trapping rain: two-pointer O(1) space — mention as follow-up
- ✓ Generalise: prefix MAX, prefix MIN, prefix AND/OR — same two-pass idea

CP CONTEST TRICKS

- ★ Prefix max + suffix max = range max query without segment tree (static array only)
- ★ Circular array: duplicate array (2N) and apply prefix/suffix on the extended version
- ★ Prefix AND / OR: useful for bitwise range queries in O(1)
- ★ Suffix sums often appear in DP: dp[i] uses dp[i+1] (suffix relationship)
- ★ 2D prefix + 2D suffix: for corner-to-corner rectangle queries

SIMILAR PROBLEMS

Problem	Platform	Relation
Product of Array Except Self	LeetCode 238	Classic prefix+suffix product

Candy	LeetCode 135	Left pass + right pass, take max
Trapping Rain Water	LeetCode 42	prefixMax + suffixMax
Find Maximum Product Subarray	LeetCode 152	Prefix + suffix max product
Sum of Distances in Tree	LeetCode 834	Prefix DFS on tree
Maximum Product of Splitted Binary Tree	LeetCode 1339	Prefix sum on tree
Shortest Distance to All Buildings	LeetCode 317	BFS + prefix counting

MERGE INTERVALS

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★■

Companies: Google, Meta, Amazon, Microsoft, Bloomberg, Uber, Stripe

RECOGNITION SIGNALS

- List of intervals, need to merge all overlapping ones
- Insert a new interval into a sorted non-overlapping list
- Find intersection of two interval lists
- Words: 'merge overlapping', 'combine intervals', 'meeting times'
- Total length covered by union of intervals

INTUITION

Sort intervals by START time. Scan linearly. If current interval START $\leq$ previous END, they overlap — merge by extending END. Otherwise, current interval is disjoint — start a new one. Two intervals $[a,b]$ and $[c,d]$ overlap when $c \leq b$.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N^2)$	$O(N)$	Learning / tiny N
Optimal	$O(N \log N)$	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Brute: repeatedly check all pairs —  $O(N^2)$ 
// Sort + linear scan is already optimal — shown in optimal below
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Merge Intervals (LeetCode 56):  $O(N \log N)$ 
vector<vector<int>> merge(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end()); // sort by start time
    vector<vector<int>> merged;

    for (auto& iv : intervals) {
        if (merged.empty() || iv[0] > merged.back()[1]) {
            merged.push_back(iv); // no overlap: new interval
        } else {
```

```

merged.back()[1] = max(merged.back()[1], iv[1]); // overlap: extend end
}
}
return merged;
}

// Insert Interval (LeetCode 57): 3-phase approach
vector<vector<int>> insert(vector<vector<int>>& intervals, vector<int> newIv) {
vector<vector<int>> result;
int i = 0, n = intervals.size();

// Phase 1: add all intervals BEFORE newIv (end < newIv.start)
while (i < n && intervals[i][1] < newIv[0])
result.push_back(intervals[i++]);

// Phase 2: merge all overlapping with newIv
while (i < n && intervals[i][0] <= newIv[1]) {
newIv[0] = min(newIv[0], intervals[i][0]);
newIv[1] = max(newIv[1], intervals[i][1]);
i++;
}
result.push_back(newIv);

// Phase 3: add all intervals AFTER newIv
while (i < n) result.push_back(intervals[i++]);
return result;
}

// Interval Intersection (LeetCode 986): two sorted lists
vector<vector<int>> intervalIntersection(vector<vector<int>>& A, vector<vector<int>>& B) {
vector<vector<int>> result;
int i = 0, j = 0;
while (i < (int)A.size() && j < (int)B.size()) {
int lo = max(A[i][0], B[j][0]);
int hi = min(A[i][1], B[j][1]);
if (lo <= hi) result.push_back({lo, hi}); // valid intersection
if (A[i][1] < B[j][1]) i++; // advance list with earlier ending interval
else j++;
}
return result;
}

```

DRY RUN — Merge: [[1,3],[2,6],[8,10],[15,18]]

interval	merged.back())	iv[0]>back[1]?	action	merged result
----------	--------------------	----------------	--------	---------------

[1,3]	empty	—	push [1,3]	[[1,3]]
[2,6]	[1,3]	2>3? NO	extend: back[1]=max(3,6)=6	[[1,6]]
[8,10]	[1,6]	8>6? YES	push [8,10]	[[1,6],[8,10]]
[15,18]	[8,10]	15>10? YES	push [15,18]	[[1,6],[8,10],[15,18]] DONE

COMMON MISTAKES + EDGE CASES

- ✗ Not sorting before merging — overlap check only works on START-sorted intervals
- ✗ Overlap condition: $iv[0] \leq merged.back()[1]$, NOT $<$ (touching endpoints = overlapping)
- ✗ Extending end: must take MAX of both ends, not just new interval's end
- ✗ Insert Interval: 3 distinct phases — before/overlapping/after — keep them separate
- ✗ Intersection: advance the pointer with SMALLER end time (earlier finishing interval)

INTERVIEW TIPS

- ✓ Say: 'Sort by start. If current start $\leq$ previous end, merge by extending end.'
- ✓ Insert Interval: three clean phases, say them explicitly — shows structured thinking
- ✓ Intersection: exactly like merging two sorted arrays with two pointers
- ✓ Follow-up: minimum rooms needed = max overlap at any point (sweep line / priority queue)
- ✓ Follow-up: total union length = sum of (end - start) for each merged interval

CP CONTEST TRICKS

- ★ After merging, binary search on merged intervals for $O(\log N)$ point-in-interval queries
- ★ Difference array is $O(N+Q)$ alternative to sweep line for integer coordinates
- ★ For K-way interval merge: priority queue (min-heap by start time)
- ★ Union length: merge then compute $\sum(iv[1]-iv[0])$ for each merged interval
- ★ Coordinate compress + sweep line for large coordinate ranges

SIMILAR PROBLEMS

Problem	Platform	Relation
Merge Intervals	LeetCode 56	Core merge pattern
Insert Interval	LeetCode 57	3-phase insertion
Interval List Intersections	LeetCode 986	Two-pointer on sorted
Meeting Rooms	LeetCode 252	Any overlap? — sort + check
Meeting Rooms II	LeetCode 253	Max concurrent rooms
Non-overlapping Intervals	LeetCode 435	Sort by end + greedy
My Calendar I/II/III	LeetCode 729/731/732	Interval insertion variants

SWEEP LINE

FAANG FREQ

★★★★■

LEETCODE

★★★★■

CODEFORCES

★★★★★

Companies: Google, Amazon, Bloomberg, Codeforces, Palantir

RECOGNITION SIGNALS

- Count maximum simultaneous events at any point in time
- The Skyline Problem — building outlines
- Area under union of rectangles
- Words: 'count active', 'max simultaneous', 'at any point in time'
- Event-based problems: each interval generates a start and end event

INTUITION

Convert each interval into two events: +1 at start, -1 at end. Sort all events by time. Sweep left to right tracking running count. Maximum running count = maximum overlap. General framework: anything that changes state at discrete time points.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N^2)$	$O(1)$	Learning / tiny N
Optimal	$O(N \log N)$	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Max overlap brute: check all pairs  $O(N^2)$ 
int maxOverlap_brute(vector<vector<int>>& intervals) {
    int maxCnt = 0;
    for (auto& [s, e] : intervals) {
        int cnt = 0;
        for (auto& [s2, e2] : intervals)
            if (s2 <= s && s < e2) cnt++;
        maxCnt = max(maxCnt, cnt);
    }
    return maxCnt;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```

// Sweep Line: max concurrent events —  $O(N \log N)$ 
int maxOverlapSweep(vector<vector<int>>& intervals) {
    vector<pair<int,int>> events;
    for (auto& iv : intervals) {
        events.push_back({iv[0], 1}); // +1: interval starts
        events.push_back({iv[1], -1}); // -1: interval ends
    }
    // At same time: process ends BEFORE starts (avoid counting closed+opening as overlap)
    sort(events.begin(), events.end(), [](auto& a, auto& b){
        return a.first != b.first ? a.first < b.first : a.second < b.second;
    });
    int curr = 0, maxCnt = 0;
    for (auto& [time, delta] : events) {
        curr += delta;
        maxCnt = max(maxCnt, curr);
    }
    return maxCnt;
}

// The Skyline Problem (LeetCode 218): sweep + multiset
vector<vector<int>> getSkyline(vector<vector<int>>& buildings) {
    vector<pair<int,int>> events;
    for (auto& b : buildings) {
        events.push_back({b[0], -b[2]}); // start: store as negative height
        events.push_back({b[1], b[2]}); // end: positive height
    }
    sort(events.begin(), events.end()); // start events come first (negative < positive)

    multiset<int, greater<int>> heights = {0}; // active heights, descending
    vector<vector<int>> result;
    int prevMax = 0;

    for (auto& [x, h] : events) {
        if (h < 0) heights.insert(-h); // building starts
        else heights.erase(heights.find(h)); // building ends

        int currMax = *heights.begin();
        if (currMax != prevMax) { // skyline changed
            result.push_back({x, currMax});
            prevMax = currMax;
        }
    }
    return result;
}

```

DRY RUN — Max overlap: intervals=[[1,4],[2,5],[3,6]]

time	event	running count	maxCnt
1	+1 (start [1,4])	1	1
2	+1 (start [2,5])	2	2
3	+1 (start [3,6])	3	3
4	-1 (end [1,4])	2	3
5	-1 (end [2,5])	1	3
6	-1 (end [3,6])	0	3 -> answer=3

COMMON MISTAKES + EDGE CASES

✗ Same timestamp: process ENDS before STARTS — prevents counting simultaneous end+start as overlap

✗ Skyline: encode starts as NEGATIVE heights — sorting naturally puts starts before ends at same x

✗ Skyline: use multiset (not unordered_set) — need to erase ONE copy of duplicate height

✗ Always initialise Skyline height set with 0 — when all buildings end, height returns to 0

✗ Difference array is faster $O(N+Q)$ for integer coordinates — better when coordinates are small

INTERVIEW TIPS

✓ Say: 'Each interval becomes two events. Sort. Track running count. Max = answer.'

✓ Skyline: multiset acts as a priority queue that also supports deletion — explain this

✓ Mention when to use diff array vs sweep line: integers -> diff array, general -> sweep line

✓ Follow-up: total covered length = integrate (count > 0) over time

✓ Follow-up: area under rectangles = 2D sweep line (advanced)

CP CONTEST TRICKS

★ Coordinate compress before sweep when coordinates are large

★ Event-based simulation is a general framework for state changes over time

★ CF: sweep line + segment tree for complex range overlap counting

★ For area union: sweep by x, maintain y-interval union length with segment tree

★ Offline processing: treat queries as events alongside data

SIMILAR PROBLEMS

Problem	Platform	Relation
Meeting Rooms II	LeetCode 253	Max concurrent events
The Skyline Problem	LeetCode 218	Sweep + multiset
My Calendar III	LeetCode 732	Max overlap with updates
Number of Flowers in Full Bloom	LeetCode 2251	Sweep + binary search
Corporate Flight Bookings	LeetCode 1109	Difference array variant
Rectangle Area II	LeetCode 850	2D sweep line
Minimum Interval Query	LeetCode 1851	Offline sweep + heap

BUCKET SORT

FAANG FREQ

★★★★■

LEETCODE

★★★★■

CODEFORCES

★★★★■

Companies: Google, Amazon, Meta, Bloomberg, Goldman Sachs

RECOGNITION SIGNALS

- Top-K frequent elements in $O(N)$ — bucket by frequency
- Sort elements when value range is known and bounded
- Maximum gap between consecutive elements after sorting
- Words: 'sort in $O(N)$ ', 'top K by frequency', 'H-index'
- When comparison-based sort is too slow and value range is usable

INTUITION

When values fall in range $[0, \text{maxVal}]$, create BUCKETS for sub-ranges. Place each element into its bucket. Collect in order. Frequency bucket trick: $\text{bucket}[\text{freq}] = \text{list of elements with that frequency}$. Scan from high freq to low $\rightarrow O(N)$ Top-K without any heap.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N \log N)$	$O(N)$	Learning / tiny N
Optimal	$O(N)$	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Comparison sort:  $O(N \log N)$  — we want  $O(N)$ 
// Bucket sort exploits known value range to beat comparison lower bound
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Top K Frequent via Bucket Sort:  $O(N)$ 
vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int,int> freq;
    for (int x : nums) freq[x]++;

    // Bucket indexed by frequency (max freq = N)
    int n = nums.size();
    vector<vector<int>> bucket(n + 1);
    for (auto& [val, cnt] : freq)
```

```

bucket[cnt].push_back(val);

vector<int> result;
for (int i = n; i >= 1 && (int)result.size() < k; i--)
for (int x : bucket[i])
if ((int)result.size() < k) result.push_back(x);
return result;
}

// Maximum Gap (LeetCode 164): O(N) using bucket/pigeonhole
int maximumGap(vector<int>& nums) {
int n = nums.size();
if (n < 2) return 0;
int mn = *min_element(nums.begin(), nums.end());
int mx = *max_element(nums.begin(), nums.end());
if (mn == mx) return 0;

// Bucket size: at least 1, ceil((mx-mn)/(n-1))
int bucketSize = max(1, (mx - mn) / (n - 1));
int bucketCount = (mx - mn) / bucketSize + 1;
vector<int> bMin(bucketCount, INT_MAX), bMax(bucketCount, INT_MIN);

for (int x : nums) {
int idx = (x - mn) / bucketSize;
bMin[idx] = min(bMin[idx], x);
bMax[idx] = max(bMax[idx], x);
}

// Max gap MUST be between adjacent non-empty buckets (pigeonhole)
int maxGap = 0, prevMax = mn;
for (int i = 0; i < bucketCount; i++) {
if (bMin[i] == INT_MAX) continue; // empty bucket
maxGap = max(maxGap, bMin[i] - prevMax);
prevMax = bMax[i];
}
return maxGap;
}

// H-Index (LeetCode 274): count sort
int hIndex(vector<int>& citations) {
int n = citations.size();
vector<int> cnt(n + 1, 0);
for (int c : citations) cnt[min(c, n)]++;
int total = 0;
for (int h = n; h >= 0; h--) {
total += cnt[h];

```



```

if (total >= h) return h;
}
return 0;
}

```

DRY RUN — Top-K Frequent: nums=[1,1,1,2,2,3], k=2

freq map	bucket[3]	bucket[2]	bucket[1]	scan i=3	scan i=2	result
{1:3,2:2,3:1}	[1]	[2]	[3]	add 1 → [1]	add 2 → [1,2]	[1,2] ✓ k=2 reached

COMMON MISTAKES + EDGE CASES

- ✗ Bucket array size must be N+1 — frequency can be at most N (all elements same)
- ✗ Maximum Gap: bucket size at least 1 to avoid division by zero
- ✗ Maximum Gap: max gap is always BETWEEN buckets by pigeonhole — never within a bucket
- ✗ Empty bucket check in Maximum Gap: skip if bMin[i] == INT_MAX
- ✗ H-Index: cap citations at N (citations > N treated as N) — otherwise array OOB

INTERVIEW TIPS

- ✓ Top-K with bucket sort: $O(N)$ beats $O(N \log K)$ heap — impressive $O(N)$ solution
- ✓ Maximum Gap: explain pigeonhole principle clearly — max gap must be between buckets
- ✓ Bucket sort requires KNOWN value range — always verify this before applying
- ✓ For Top-K: mention both approaches: heap $O(N \log K)$ and bucket $O(N)$ — discuss tradeoffs
- ✓ H-Index: bucket sort makes it $O(N)$ vs $O(N \log N)$ sort-based approach

CP CONTEST TRICKS

- ★ Frequency bucket sort: $O(N)$ Top-K — faster constant than heap for large inputs
- ★ Radix sort: sort by each digit LSD to MSD — $O(N*d)$ where d = number of digits
- ★ Counting sort for values in $[0, K]$: $O(N+K)$ — great when K is small
- ★ Combine with coordinate compression: compress values -> bucket sort on compressed
- ★ Pigeonhole principle: $N+1$ items in N buckets -> at least one bucket has 2 items

SIMILAR PROBLEMS

Problem	Platform	Relation
Top K Frequent Elements	LeetCode 347	Frequency bucket sort
Maximum Gap	LeetCode 164	Bucket + pigeonhole
H-Index	LeetCode 274	Count sort + scan
Sort Characters By Frequency	LeetCode 451	Bucket on char freq
Relative Sort Array	LeetCode 1122	Custom bucket sort
Kth Largest Element	LeetCode 215	QuickSelect or bucket
Sort Array by Parity	LeetCode 905	2-bucket partition

BINARY SEARCH ON ARRAYS

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★★

Companies: Google, Meta, Amazon, Microsoft, Apple, Bloomberg, Uber

RECOGNITION SIGNALS

- Array is sorted OR problem has a monotonic feasibility condition
- Words: 'find minimum that satisfies', 'maximize the minimum', 'feasible'
- Search in rotated sorted array
- Find first/last occurrence of element
- Binary search on ANSWER VALUE (not on array index)

INTUITION

Binary search works on any MONOTONIC predicate. If predicate(X) goes False...False...True...True, binary search finds the boundary in $O(\log N)$. For 'maximize minimum' or 'minimize maximum': binary search on the answer. Check if a given answer value is feasible in $O(N)$. Total: $O(N \log(\maxVal))$.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N)$	$O(1)$	Learning / tiny N
Optimal	$O(\log N)$ search, $O(N \log \maxVal)$ binary search on answer	$O(1)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Linear search: O(N)
int search_brute(vector<int>& nums, int target) {
    for (int i = 0; i < (int)nums.size(); i++)
        if (nums[i] == target) return i;
    return -1;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Standard Binary Search Template
int binarySearch(vector<int>& nums, int target) {
```

```

int lo = 0, hi = (int)nums.size() - 1;
while (lo <= hi) {
int mid = lo + (hi - lo) / 2; // NEVER (lo+hi)/2 - can overflow
if (nums[mid] == target) return mid;
else if (nums[mid] < target) lo = mid + 1;
else hi = mid - 1;
}
return -1;
}

```

```

// Search in Rotated Sorted Array (LeetCode 33)
int searchRotated(vector<int>& nums, int target) {
int lo = 0, hi = (int)nums.size() - 1;
while (lo <= hi) {
int mid = lo + (hi - lo) / 2;
if (nums[mid] == target) return mid;
if (nums[lo] <= nums[mid]) { // left half is sorted
if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
else lo = mid + 1;
} else { // right half is sorted
if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
else hi = mid - 1;
}
}
return -1;
}

```

```

// Binary Search on Answer: Koko Eating Bananas (LeetCode 875)
bool canEat(vector<int>& piles, int speed, int h) {
long long hrs = 0;
for (int p : piles) hrs += (p + speed - 1) / speed; // ceil division
return hrs <= h;
}

int minEatingSpeed(vector<int>& piles, int h) {
int lo = 1, hi = *max_element(piles.begin(), piles.end());
while (lo < hi) { // find LEFTMOST valid speed
int mid = lo + (hi - lo) / 2;
if (canEat(piles, mid, h)) hi = mid; // valid: try smaller
else lo = mid + 1; // invalid: need more
}
return lo;
}

```

```

// Find First and Last Position (LeetCode 34)
int findFirst(vector<int>& nums, int target) {
int lo = 0, hi = (int)nums.size()-1, ans = -1;

```

```

while (lo <= hi) {
    int mid = lo + (hi-lo)/2;
    if (nums[mid] == target) { ans = mid; hi = mid-1; } // keep searching left
    else if (nums[mid] < target) lo = mid+1;
    else hi = mid-1;
}
return ans;
}

```

DRY RUN — Search Rotated: nums=[4,5,6,7,0,1,2], target=0

lo	hi	mid	nums[mid]	left sorted?	target in left?	move
0	6	3	7	4<=7 YES	4<=0<7? NO	lo=4
4	6	5	1	0<=1 YES	0<=0<1? YES	hi=4
4	4	4	0	—	nums[4]==0	FOUND index=4 ✓

BINARY SEARCH ON ANSWER — TEMPLATE

Use when: 'Find minimum X such that condition(X) is True.' Condition must be monotonic.

```

// Leftmost True: False False ... True True
int lo = minPossible, hi = maxPossible;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (condition(mid)) hi = mid; // valid: try smaller
    else lo = mid + 1; // invalid: need larger
}
return lo; // lo == hi == answer

// Rightmost True: True True ... False False
// Use: if (condition(mid)) lo = mid+1; else hi = mid;
// Answer: hi (or lo-1 after loop)

```

Classic examples: Koko bananas, ship packages, split array, minimum days to bloom.

COMMON MISTAKES + EDGE CASES

- ✗ Using $(lo+hi)/2$ — overflows for large lo and hi. ALWAYS use $lo + (hi-lo)/2$
- ✗ $lo \leq hi$ for EXACT search. $lo < hi$ for BOUNDARY search (finding answer)
- ✗ Rotated array: check which half is sorted FIRST, then check if target is in that half
- ✗ Answer binary search: $hi = mid$ (not $mid-1$) and $lo = mid+1$ to converge correctly
- ✗ Off-by-one in feasibility: ceil division for splitting problems — use $(a+b-1)/b$

INTERVIEW TIPS

- ✓ Say: 'Binary search requires monotonic condition — let me verify that holds here.'
- ✓ Binary search on answer is the key insight for many hard problems — name it explicitly
- ✓ Rotated array: determine sorted half first — that's the entire trick
- ✓ STL: `lower_bound` returns first element $\geq x$, `upper_bound` returns first $> x$

✓ Follow-up: median of two sorted arrays — $O(\log(\min(M,N)))$ binary search

CP CONTEST TRICKS

- ★ Binary search on answer: $O(N \log \text{maxVal})$ — very powerful for optimization problems
- ★ Condition must be monotonic — always prove this before implementing
- ★ Real-valued binary search: run exactly 100 iterations instead of $\text{lo} < \text{hi}$
- ★ Fractional programming: maximize a/b — binary search on ratio value
- ★ Parallel binary search: answer multiple binary search queries simultaneously in $O((N+Q) \log N)$

SIMILAR PROBLEMS

Problem	Platform	Relation
Binary Search	LeetCode 704	Standard template
Search in Rotated Sorted Array	LeetCode 33	Rotated binary search
Koko Eating Bananas	LeetCode 875	Binary search on answer
Capacity to Ship in D Days	LeetCode 1011	Binary search on answer
Split Array Largest Sum	LeetCode 410	Binary search on answer hard
Find Minimum in Rotated Array	LeetCode 153	Rotated minimum
Median of Two Sorted Arrays	LeetCode 4	$O(\log \min(M,N))$ hard

COUNTING SORT

FAANG FREQ

★★★★■

LEETCODE

★★★★■

CODEFORCES

★★★★■

Companies: Google, Amazon, competitive programming

RECOGNITION SIGNALS

- Values in small known range (0 to K where K is manageable)
- Need stable sort or exact $O(N+K)$ sort
- Character/digit frequency sorting
- Words: 'sort characters by frequency', 'small range values'
- When $K \ll N$ and you need $O(N)$ sort

INTUITION

Count occurrences of each value in $O(N)$. Then reconstruct sorted array by iterating counts. Total time $O(N+K)$ where K is value range. Stable variant: accumulate counts to get final positions.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N \log N)$	$O(1)$	Learning / tiny N
Optimal	$O(N+K)$	$O(K)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Comparison sort:  $O(N \log N)$ 
sort(arr.begin(), arr.end());
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Counting Sort:  $O(N + K)$ 
vector<int> countingSort(vector<int>& arr, int maxVal) {
    vector<int> count(maxVal + 1, 0);
    for (int x : arr) count[x]++; // count occurrences
    vector<int> result;
    for (int v = 0; v <= maxVal; v++)
        while (count[v]-- > 0)
            result.push_back(v);
    return result;
}
```

```

// Sort Chars by Frequency (LeetCode 451)
string frequencySort(string s) {
unordered_map<char,int> freq;
for (char c : s) freq[c]++;
// Bucket by frequency
vector<vector<char>> bucket(s.size() + 1);
for (auto& [c, f] : freq) bucket[f].push_back(c);
string result = "";
for (int i = s.size(); i >= 1; i--)
for (char c : bucket[i])
result += string(i, c); // repeat char i times
return result;
}

// Radix Sort: sort integers by each digit O(N * d)
void radixSort(vector<int>& arr) {
int maxVal = *max_element(arr.begin(), arr.end());
for (int exp = 1; maxVal / exp > 0; exp *= 10) {
vector<int> output(arr.size());
int count[10] = {0};
for (int x : arr) count[(x / exp) % 10]++;
for (int i = 1; i < 10; i++) count[i] += count[i-1]; // prefix sums
for (int i = arr.size()-1; i >= 0; i--) { // stable: right to left
int digit = (arr[i] / exp) % 10;
output[--count[digit]] = arr[i];
}
arr = output;
}
}

```

DRY RUN — countingSort: arr=[3,1,4,1,5,9,2,6], maxVal=9

Step	count array (index=value)	result
Count	[0,2,1,1,1,1,1,0,0,1]	—
Reconstruct	iterate v=0..9, repeat count[v] times	[1,1,2,3,4,5,6,9]
Done	—	sorted in O(N+K)=O(17) ✓

COMMON MISTAKES + EDGE CASES

- ✗ Count array size: maxVal+1 (not maxVal), 0-indexed
- ✗ Negative values: shift all values by min value before counting, shift back after
- ✗ Radix sort must be stable — process elements right-to-left in reconstruction
- ✗ Counting sort not suitable when K >> N (wastes memory and time on empty buckets)

INTERVIEW TIPS

- ✓ Mention $O(N+K)$ — only beats comparison sort when K is small relative to N
- ✓ Stable counting sort: use prefix sums of count array for exact output positions
- ✓ Radix sort: $O(N*d)$ where d = number of digits — useful for large N with bounded int values
- ✓ Character sorting (a-z): $K=26$, always $O(N)$ — use this for anagram/string problems

CP CONTEST TRICKS

- ★ For values $0..1e6$: counting sort array of size $1e6+1$ — 4MB, acceptable in most problems
- ★ Frequency array `int[26]` for lowercase letters: 10x faster than `unordered_map`
- ★ Radix sort beats comparison sort for large arrays of bounded integers
- ★ Counting sort as a subroutine in radix sort (LSD radix sort)

SIMILAR PROBLEMS

Problem	Platform	Relation
Sort Colors	LeetCode 75	3-value counting sort
Relative Sort Array	LeetCode 1122	Custom order counting sort
Sort Characters By Frequency	LeetCode 451	Frequency bucket sort
Top K Frequent Elements	LeetCode 347	Frequency bucket variant
Maximum Gap	LeetCode 164	Radix or bucket sort
Largest Number	LeetCode 179	Custom comparison sort

MODULAR ARITHMETIC TRICKS

FAANG FREQ

★★★★■

LEETCODE

★★★★■

CODEFORCES

★★★★★

Companies: Google, Amazon, Codeforces, competitive programming

RECOGNITION SIGNALS

- Answer must be returned modulo 10^9+7 (or 10^9+9)
- Counting combinations, permutations, or paths
- Words: 'return answer modulo', 'number of ways', 'count paths modulo'
- Power/exponent computations where result is huge
- Cyclic patterns or group theory in arrays

INTUITION

When numbers explode, use modular arithmetic. $(a + b) \% M = (a \% M + b \% M) \% M$. Same for multiply. Division requires modular inverse: $a/b \% M = a * b^{(M-2)} \% M$ (for prime M). Fast power (binary exponentiation): compute $a^b \bmod M$ in $O(\log b)$ steps.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N)$	$O(1)$	Learning / tiny N
Optimal	$O(\log N)$ for power, $O(N)$ for factorial precompute	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Naive power: too slow for large exponents
long long power_brute(long long base, long long exp, long long mod) {
    long long result = 1;
    for (long long i = 0; i < exp; i++)
        result = result * base % mod; // exp can be 1e18 -> impossible
    return result;
}
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Fast Power (Binary Exponentiation):  $O(\log N)$ 
long long fastPow(long long base, long long exp, long long mod) {
```

```

long long result = 1;
base %= mod;
while (exp > 0) {
if (exp & 1) result = result * base % mod; // odd: multiply in current base
base = base * base % mod; // square the base
exp >>= 1; // halve the exponent
}
return result;
}

// Modular Inverse (Fermat's Little Theorem): a^(MOD-2) % MOD
// Requires MOD to be PRIME
long long modInverse(long long a, long long mod) {
return fastPow(a, mod - 2, mod);
}

// Precompute factorials for nCr queries – O(N) precompute, O(1) per query
const long long MOD = 1e9 + 7;
const int MAXN = 1e6 + 5;
long long fact[MAXN], inv_fact[MAXN];
void precompute(int n) {
fact[0] = 1;
for (int i = 1; i <= n; i++) fact[i] = fact[i-1] * i % MOD;
inv_fact[n] = modInverse(fact[n], MOD);
for (int i = n-1; i >= 0; i--) inv_fact[i] = inv_fact[i+1] * (i+1) % MOD;
}
long long nCr(int n, int r) {
if (r < 0 || r > n) return 0;
return fact[n] * inv_fact[r] % MOD * inv_fact[n-r] % MOD;
}

// Safe subtraction with mod
long long subMod(long long a, long long b, long long mod) {
return (a - b % mod + mod) % mod; // add mod before % to prevent negative
}

```

DRY RUN — fastPow(2, 10, 1e9+7)

exp (binary)	base	exp & 1?	result	next base
10 = 1010	2	0 (no)	1	4
5 = 0101	4	1 (yes)	4	16
2 = 0010	16	0 (no)	4	256
1 = 0001	256	1 (yes)	4*256=1024	65536
0	—	done	1024	2^10=1024 ✓

COMMON MISTAKES + EDGE CASES

- ✗ Intermediate overflow: $(a * b) \% \text{MOD}$ needs a and b both $< \text{MOD}$ — use long long
- ✗ Correct: $(a \% \text{MOD} * (b \% \text{MOD})) \% \text{MOD}$ — mod BOTH operands before multiplying
- ✗ Subtraction can go negative: ALWAYS use $(a - b + \text{MOD}) \% \text{MOD}$
- ✗ Division by X: use modular inverse $\text{modInverse}(X, \text{MOD})$, NEVER $a/b \% \text{MOD}$ directly
- ✗ Fermat's theorem requires PRIME modulus — 10^9+7 and 10^9+9 are both prime

INTERVIEW TIPS

- ✓ Always use long long for ALL intermediate calculations, even if final answer fits in int
- ✓ Say: 'I'll use fast exponentiation for $O(\log N)$ power computation'
- ✓ Precompute factorials when multiple nCr queries needed — explain $O(N)$ precompute, $O(1)$ query
- ✓ Mention both MOD choices: 10^9+7 (most common) and 10^9+9 (second choice for double hash)
- ✓ Subtraction rule is the most commonly forgotten — always add MOD before taking %

CP CONTEST TRICKS

- ★ $nCr \bmod \text{prime}$: $O(N)$ precompute then $O(1)$ per query — essential for combinatorics CF problems
- ★ Matrix exponentiation: compute linear recurrences (like Fibonacci) in $O(k^3 \log n)$
- ★ Lucas' theorem: $nCr \bmod p$ for small prime p (when n can be huge)
- ★ Chinese Remainder Theorem: combine results modulo different primes
- ★ For non-prime MOD: use extended Euclidean algorithm for modular inverse

SIMILAR PROBLEMS

Problem	Platform	Relation
Pow(x,n)	LeetCode 50	Fast exponentiation
Count Vowel Strings	LeetCode 1641	DP with MOD
Unique Paths	LeetCode 62	Combinatorics $C(m+n-2, m-1)$
Count All Possible Routes	LeetCode 1575	DP + MOD
Number of Ways to Reorder Array	LeetCode 1569	Combinatorics + MOD
Count Ways to Build Good Strings	LeetCode 2466	DP + MOD
Fibonacci Number	LeetCode 509	Matrix exponentiation

BITMASKING BASICS

FAANG FREQ

★★★★■

LEETCODE

★★★★■

CODEFORCES

★★★★★

Companies: Google, Meta, Amazon, Codeforces

RECOGNITION SIGNALS

- Enumerate all subsets of a small set ($N \leq 20$)
- State compression in DP — visited cities, used items, selected elements
- Words: 'subsets', 'all combinations', ' $N \leq 20$ with bitmask DP'
- Travelling Salesman Problem (TSP) with small N
- Shortest path visiting all nodes (bitmask BFS/DP)

INTUITION

An integer's bits represent a SUBSET: bit $i = 1$ means element i is included. For N elements, 2^N subsets, each is an integer 0 to $2^N - 1$. Enumerate with `for(mask=0; mask<(1<<N); mask++)`. Core operations: check bit, set bit, clear bit, count bits.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(2^N)$ (unavoidable for subset enumeration)	$O(N)$	Learning / tiny N
Optimal	$O(2^N * N)$ for subset, $O(2^N * N^2)$ for TSP	$O(2^N * N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Enumerate subsets with nested loops (only works for fixed N)
// For N=3: 3 nested loops – not scalable
// Use bitmask for variable N
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Enumerate all  $2^N$  subsets with bitmask
void enumerateSubsets(vector<int>& arr) {
    int n = arr.size(); // must be <= 20 for practical use
    for (int mask = 0; mask < (1 << n); mask++) {
```

```

vector<int> subset;
for (int i = 0; i < n; i++)
if ((mask >> i) & 1) // is bit i set?
subset.push_back(arr[i]);
// process subset here
}
}

// TSP Bitmask DP:  $O(2^N * N^2)$  – classic for  $N \leq 20$ 
int tsp(vector<vector<int>>& dist) {
int n = dist.size();
const int INF = 1e9;
// dp[mask][i] = min cost to visit exactly the cities in mask, ending at city i
vector<vector<int>> dp(1 << n, vector<int>(n, INF));
dp[1][0] = 0; // start at city 0, only city 0 visited (mask = 0001)
for (int mask = 1; mask < (1 << n); mask++) {
for (int u = 0; u < n; u++) {
if (dp[mask][u] == INF) continue;
if (!(mask >> u) & 1) continue; // u not in mask
for (int v = 0; v < n; v++) {
if ((mask >> v) & 1) continue; // v already visited
int newMask = mask | (1 << v);
dp[newMask][v] = min(dp[newMask][v], dp[mask][u] + dist[u][v]);
}
}
}
int full = (1 << n) - 1, ans = INF;
for (int u = 1; u < n; u++) ans = min(ans, dp[full][u] + dist[u][0]);
return ans;
}

// Common bit tricks reference:
bool isPowerOf2(int n) { return n > 0 && (n & (n-1)) == 0; }
int popcount(int n) { return __builtin_popcount(n); }
int lowestSetBit(int n) { return n & (-n); } // isolate rightmost 1-bit
int clearLowest(int n) { return n & (n-1); } // clear rightmost 1-bit
bool kthBitSet(int n, int k){ return (n >> k) & 1; }
int setKthBit(int n, int k){ return n | (1 << k); }
int clearKthBit(int n,int k){ return n & ~(1 << k);}

```

DRY RUN — All subsets of [a,b,c] (N=3, masks 0-7)

mask	binary	bit2(c)	bit1(b)	bit0(a)	subset
0	000	0	0	0	{}
1	001	0	0	1	{a}
2	010	0	1	0	{b}

3	011	0	1	1	{a,b}
4	100	1	0	0	{c}
5	101	1	0	1	{a,c}
6	110	1	1	0	{b,c}
7	111	1	1	1	{a,b,c}

BIT TRICKS CHEAT SHEET

Operation	Code	Notes
Check bit k	$(n \gg k) \& 1$	Returns 0 or 1
Set bit k	$n \mid (1 \ll k)$	Turns bit k on
Clear bit k	$n \& \sim(1 \ll k)$	Turns bit k off
Toggle bit k	$n \wedge (1 \ll k)$	Flips bit k
Clear lowest 1	$n \& (n-1)$	Removes rightmost set bit
Isolate lowest 1	$n \& (-n)$	Only rightmost 1 remains
Count set bits	<code>__builtin_popcount(n)</code>	Number of 1s
Is power of 2	$n > 0 \&\& (n \& (n-1)) == 0$	Exactly one bit set
Enumerate submasks	$m = (m-1) \& \text{mask};$	All subsets of mask (loop)
Full mask (N bits)	$(1 \ll N) - 1$	All N bits = 1

COMMON MISTAKES + EDGE CASES

✗ $N > 25$: $2^{25} = 33\text{M}$ — barely ok. $N > 30$: $2^{30} = 1\text{B}$ — too slow

✗ $1 \ll N$ when $N=32$: undefined behavior in C++ — use $1LL \ll N$ for large N

✗ Full mask: $(1 \ll n) - 1$ not $(1 < n)$ — easy to confuse

✗ Submask iteration: `for(m=mask; m>0; m=(m-1)&mask;)` — misses empty set (mask=0)

✗ TSP: starting mask = 1 (only city 0 visited), not 0

INTERVIEW TIPS

✓ Say: 'Bit i represents whether element i is in the subset — $O(1)$ subset operations'

✓ Always check $N \leq 20$ before using bitmask DP — mention this constraint check

✓ TSP is the classic bitmask DP — $O(2^N \cdot N^2)$ time, $O(2^N \cdot N)$ space

✓ For subset sum: bitmask DP if $N \leq 20$, regular DP if values are small

✓ Bit tricks (popcount, isolate lowest) show C++ depth knowledge

CP CONTEST TRICKS

★ Submask enumeration: total iterations = 3^N (sum over all masks of $2^{\text{popcount}(\text{mask})}$)

★ SOS DP (Sum over Subsets): $f[\text{mask}] = \text{sum of all submask values}$ in $O(N \cdot 2^N)$

★ Profile DP on grids: bitmask = current row state, transitions row by row

★ Meet in the middle: split $N=2 \cdot (N/2)$, enumerate halves separately — $O(2^{N/2})$

★ Bit manipulation in XOR problems: contribution per bit, linearity of XOR

SIMILAR PROBLEMS

Problem	Platform	Relation
Subsets	LeetCode 78	Enumerate all subsets
Subsets II	LeetCode 90	Subsets with duplicates
Shortest Path Visiting All Nodes	LeetCode 847	BFS + bitmask state
Maximum AND Sum	LeetCode 2172	Bitmask DP
Smallest Sufficient Team	LeetCode 1125	Bitmask DP hard
Partition into K Equal Subsets	LeetCode 698	Bitmask + backtracking
Minimum Cost to Connect Points	LeetCode 1584	MST on complete graph

GREEDY + HASHING

FAANG FREQ

★★★★■

LEETCODE

★★★★★

CODEFORCES

★★★★■

Companies: Google, Amazon, Meta, Microsoft, Stripe

RECOGNITION SIGNALS

- Rearrange or schedule elements to satisfy constraints
- Most/least frequent element drives the arrangement
- Words: 'reorganize', 'rearrange', 'schedule tasks', 'at least K apart'
- Group elements sharing a property (anagrams, same key)
- Greedy selection guided by frequency from HashMap

INTUITION

Use HashMap to get frequency or grouping info in $O(N)$. Then apply greedy decisions driven by that information. Task Scheduler: most frequent task is the bottleneck. The answer is $\max(\text{total tasks}, (\text{maxFreq}-1)*(n+1)+\text{countOfMaxFreq})$.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N!)$	$O(N)$	Learning / tiny N
Optimal	$O(N \log N)$ or $O(N)$	$O(N)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Task Scheduler brute: try all orderings –  $O(N! * N)$ 
// Not feasible. Use greedy formula instead.
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Task Scheduler (LeetCode 621):  $O(N)$ 
int leastInterval(vector<char>& tasks, int n) {
    int freq[26] = {0};
    for (char c : tasks) freq[c - 'A']++;
    int maxFreq = *max_element(freq, freq + 26);
    int countMax = count(freq, freq + 26, maxFreq);
    // Formula: idle slots filled greedily
    return max((int)tasks.size(), (maxFreq - 1) * (n + 1) + countMax);
}
```



```

// Reorganize String (LeetCode 767): no two adjacent same
string reorganizeString(string s) {
int freq[26] = {0};
for (char c : s) freq[c - 'a']++;
int maxFreq = *max_element(freq, freq + 26);
if (maxFreq > (s.size() + 1) / 2) return ""; // impossible
// Greedy: always place most frequent remaining char != last placed
priority_queue<pair<int, char>> pq;
for (int i = 0; i < 26; i++)
if (freq[i] > 0) pq.push({freq[i], 'a' + i});
string result = "";
while (pq.size() >= 2) {
auto [f1, c1] = pq.top(); pq.pop();
auto [f2, c2] = pq.top(); pq.pop();
result += c1; result += c2;
if (f1 - 1 > 0) pq.push({f1-1, c1});
if (f2 - 1 > 0) pq.push({f2-1, c2});
}
if (!pq.empty()) result += pq.top().second;
return result;
}

// Group Anagrams (LeetCode 49): sorted string as HashMap key
vector<vector<string>> groupAnagrams(vector<string>& strs) {
unordered_map<string, vector<string>> groups;
for (string& s : strs) {
string key = s;
sort(key.begin(), key.end()); // canonical sorted form
groups[key].push_back(s);
}
vector<vector<string>> result;
for (auto& [k, v] : groups) result.push_back(v);
return result;
}

```

DRY RUN — Task Scheduler: tasks=[A,A,A,B,B,B], n=2

maxFreq	countMax	Formula	tasks.size()	max(both)
3	2 (A and B tie)	$(3-1)*(2+1)+2=8$	6	$\max(6,8)=8$
Schedule:	A B _	A B _	A B	8 slots (2 idles) ✓

COMMON MISTAKES + EDGE CASES

X Task Scheduler formula: use $\max(\text{tasks.size()}, (\text{maxFreq}-1)*(n+1)+\text{countMax})$ — both matter

X Reorganize String: if $\text{maxFreq} > (N+1)/2$ it's impossible — check this first

X Group Anagrams: sorted string as key is $O(K \log K)$ per string — `int[26]` array is $O(K)$

✗ Task Scheduler: count tasks with MAX frequency (not just one) for countMax

✗ Priority queue: pop TWO at a time to avoid placing same char consecutively

INTERVIEW TIPS

- ✓ Task Scheduler: explain formula — 'idle slots between repeating max-freq task blocks'
- ✓ Group Anagrams: char-count array as key avoids sort — $O(K)$ per string vs $O(K \log K)$
- ✓ Reorganize String: greedy always picks top-2 most frequent — interleaves naturally
- ✓ Greedy correctness proof: exchange argument — any deviation can only worsen the result
- ✓ Follow-up: what if task has a deadline too? -> use min-heap with (deadline, freq)

CP CONTEST TRICKS

- ★ Max-heap greedy: always process most frequent first — reduces to $O(N \log K)$
- ★ Char count array `int[26]` as HashMap key: avoids sort, $O(1)$ comparison with `memcmp`
- ★ Rearrangement condition: max frequency $\leq \lceil N/2 \rceil$ for any adjacent-constraint problem
- ★ Scheduling with cooldown: the formula $(\text{maxFreq}-1)*(n+1)+\text{countMax}$ is reusable
- ★ For CF: greedy by frequency is $O(N \log N)$ — often beats brute $O(N^2)$

SIMILAR PROBLEMS

Problem	Platform	Relation
Task Scheduler	LeetCode 621	Greedy with max freq formula
Reorganize String	LeetCode 767	Greedy + max-heap
Group Anagrams	LeetCode 49	Sorted string as key
Rearrange String K Distance Apart	LeetCode 358	Greedy + priority queue
Minimum Number of Frogs Croaking	LeetCode 1419	State tracking HashMap
Least Unique Integers after K Removals	LeetCode 1481	Sort by freq greedy
Find and Replace Pattern	LeetCode 890	Bijection with two HashMaps

SPARSE FREQUENCY + OFFLINE TRICKS

FAANG FREQ

★★★★

LEETCODE

★★★★

CODEFORCES

★★★★★

Companies: Codeforces, Google, competitive programming heavy

RECOGNITION SIGNALS

- Frequency queries on subarrays with offline processing
- MO's Algorithm — block decomposition for range queries
- Range frequency queries: 'how many times X appears in [L,R]'
- Offline: can sort queries in any order for efficiency
- Words: 'frequency in range', 'count occurrences in subarray'

INTUITION

Online range frequency is hard. But OFFLINE (all queries given upfront), we can sort them by MO's order (block sort) and answer all in $O((N+Q)*\sqrt{N})$. MO's Algorithm: maintain a window $[L, R]$. When query changes L or R by 1, update counter in $O(1)$. Block-sort queries to minimize total pointer movements.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(N*Q)$	$O(1)$	Learning / tiny N
Optimal	$O((N+Q)*\sqrt{N})$	$O(N+Q)$	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Range frequency brute: O(N) per query
int freqInRange(vector<int>& arr, int L, int R, int x) {
    int cnt = 0;
    for (int i = L; i <= R; i++)
        if (arr[i] == x) cnt++;
    return cnt;
} // O(N*Q) total for Q queries
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// MO's Algorithm: O((N+Q)*sqrt(N))
// Sort queries by block to minimise pointer movements
struct Query { int l, r, idx; };
```

```

vector<int> mo(vector<int>& arr, vector<pair<int,int>>& queries) {
int n = arr.size(), q = queries.size();
int block = max(1, (int)sqrt(n));
vector<Query> qs(q);
for (int i = 0; i < q; i++) qs[i] = {queries[i].first, queries[i].second, i};

// Sort: by block of L, then by R (alternating direction for even blocks)
sort(qs.begin(), qs.end(), [&](Query& a, Query& b) {
int ba = a.l / block, bb = b.l / block;
if (ba != bb) return ba < bb;
return (ba & 1) ? (a.r > b.r) : (a.r < b.r); // Hilbert curve optimisation
});

unordered_map<int,int> freq;
int currL = 0, currR = -1, currAns = 0;
vector<int> ans(q);

auto add = [&](int pos) { if (++freq[arr[pos]] == 1) currAns++; };
auto rem = [&](int pos) { if (--freq[arr[pos]] == 0) currAns--; };

for (auto& qr : qs) {
while (currR < qr.r) add(++currR);
while (currL > qr.l) add(--currL);
while (currR > qr.r) rem(currR--);
while (currL < qr.l) rem(currL++);
ans[qr.idx] = currAns; // adapt 'ans' for your specific query type
}
return ans;
}

// Simpler: Coordinate compress + prefix frequency for static queries
// prefFreq[v][i] = how many times value v appears in arr[0..i]
// Query(L,R,v) = prefFreq[v][R] - prefFreq[v][L-1] O(1) after O(N*K) precompute

```

DRY RUN — MO's block size = $\sqrt{N}$, queries sorted by block

Query (L,R)	Block of L	Sort order	Pointer movement
(0,3)	0	Block 0, R=3	initial window [0,3]
(1,5)	0	Block 0, R=5	move R right: 4,5
(0,7)	0	Block 0, R=7	move L left, R right
(5,8)	1	Block 1	big jump in L (accepted)
(6,9)	1	Block 1, R=9	move R, small L change

COMMON MISTAKES + EDGE CASES

X Block size: use $\sqrt{N}$ — too small = too many blocks, too large = too many R moves

✗ MO's doesn't work for online queries or when array is modified

✗ Add/Remove functions must be $O(1)$ for MO's to work correctly

✗ Alternating R direction per block (Hilbert optimisation) reduces constant factor by $\sim 2\times$

✗ For range frequency: 'currAns' counts distinct elements — adapt for specific problem

INTERVIEW TIPS

✓ MO's is a CF/olympiad technique — rarely asked in FAANG but shows competitive programming depth

✓ For FAANG: simpler range frequency with sorted indices per value + binary search is $O(Q \log N)$

✓ Offline = we know all queries upfront and can answer in any order — powerful constraint

✓ Mention sqrt decomposition as a technique when both N and Q are around $1e5$

✓ Practical alternative: merge sort tree or persistent segment tree for online range freq

CP CONTEST TRICKS

★ MO's complexity: $O((N+Q)\sqrt{N})$ — optimal for many offline range query problems

★ Mo with updates: add time dimension, complexity $O(N^{5/3})$

★ Block decomposition: general technique — precompute for blocks, handle partial blocks

★ Offline processing: sort queries creatively to reuse computation

★ For small value range: prefix frequency array is $O(1)$ per query after $O(N \cdot K)$ precompute

SIMILAR PROBLEMS

Problem	Platform	Relation
Count in Range	CF Classic	MO's or prefix frequency
Powerful Array	CF 86D	Classic MO's problem
Range Mode Query	CF Classic	MO's for mode
Number of Distinct in Window	LeetCode 2461	Sliding window variant
Frequency of Most Frequent Element	LeetCode 1838	Sliding window + greedy
Subarrays with K Different Integers	LeetCode 992	Exactly K = atmost K - atmost $K-1$
Count Subarrays with Fixed Bounds	LeetCode 2444	Monotonic pointers

INTERVAL SCHEDULING (ACTIVITY SELECTION)

FAANG FREQ

★★★★★

LEETCODE

★★★★★

CODEFORCES

★★★★★

Companies: Google, Meta, Amazon, Microsoft, Bloomberg, Goldman Sachs, Uber

RECOGNITION SIGNALS

- Select maximum number of non-overlapping intervals
- Minimum number of intervals to REMOVE to make rest non-overlapping
- Words: 'maximum activities', 'non-overlapping', 'remove minimum intervals'
- Job scheduling with start/end times — pick maximum jobs
- Meeting scheduling: fit maximum meetings in a day

INTUITION

Greedy insight: always pick the interval that ends EARLIEST. Why? An interval finishing early leaves the most room for future intervals. Proof: if we swap the earliest-ending choice with any other, we can only lose future options. Sort by END time. Greedily pick every interval that starts after the last picked one ends.

COMPLEXITY OVERVIEW

Approach	Time	Space	Use When
Brute Force	$O(2^N)$	$O(N)$	Learning / tiny N
Better	$O(N^2)$	$O(1)$	DP: $dp[i] = \max$ intervals ending at i
Optimal	$O(N \log N)$	$O(1)$ greedy, $O(N)$ weighted DP	Always use in interviews/contests

BRUTE FORCE (Understand why it fails)

```
// Brute: try all subsets of non-overlapping intervals –  $O(2^N)$ 
```

```
// Enumerate all subsets, check non-overlap, find max size
```

```
// N=20 -> 1M subsets, N=30 -> 1B – completely impractical
```

OPTIMAL SOLUTION (Interview-ready C++)

```
// Activity Selection / Non-overlapping Intervals:  $O(N \log N)$ 
```

```
// Returns maximum number of non-overlapping intervals selectable
```

```
int maxNonOverlapping(vector<vector<int>>& intervals) {
```

```
// Sort by END time – core greedy insight
```

```

sort(intervals.begin(), intervals.end(),
[](auto& a, auto& b){ return a[1] < b[1]; });

int count = 1, lastEnd = intervals[0][1];
for (int i = 1; i < (int)intervals.size(); i++) {
    if (intervals[i][0] >= lastEnd) { // starts after last chosen ends
        count++;
        lastEnd = intervals[i][1];
    }
}
return count;
}

// Minimum intervals to REMOVE (LeetCode 435):
// = total - max_non_overlapping
int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    if (intervals.empty()) return 0;
    sort(intervals.begin(), intervals.end(),
[](auto& a, auto& b){ return a[1] < b[1]; });

    int keep = 1, lastEnd = intervals[0][1];
    for (int i = 1; i < (int)intervals.size(); i++) {
        if (intervals[i][0] >= lastEnd) {
            keep++;
            lastEnd = intervals[i][1];
        }
        // else: overlapping – skip this interval (count as removed)
    }
    return (int)intervals.size() - keep;
}

// Minimum Arrows to Burst Balloons (LeetCode 452):
// Each balloon is [start, end]. Arrow at x bursts all balloons where start<=x<=end.
int findMinArrowShots(vector<vector<int>>& points) {
    sort(points.begin(), points.end(),
[](auto& a, auto& b){ return a[1] < b[1]; }); // sort by end

    int arrows = 1;
    int arrowPos = points[0][1]; // shoot first arrow at first balloon's end
    for (int i = 1; i < (int)points.size(); i++) {
        if (points[i][0] > arrowPos) { // balloon starts AFTER current arrow
            arrows++;
            arrowPos = points[i][1]; // new arrow at this balloon's end
        }
        // else: current arrow already bursts this balloon too
    }
}

```

```

return arrows;
}

// Job Scheduling with Profits (LeetCode 1235): Weighted interval scheduling
// Sort by end, DP + binary search for last non-overlapping job
int jobScheduling(vector<int>& startTime, vector<int>& endTime, vector<int>& profit) {
    int n = startTime.size();
    vector<array<int,3>> jobs(n);
    for (int i = 0; i < n; i++) jobs[i] = {endTime[i], startTime[i], profit[i]};
    sort(jobs.begin(), jobs.end()); // sort by end time

    vector<int> dp(n+1, 0); // dp[i] = max profit using first i jobs
    vector<int> ends(n);
    for (int i = 0; i < n; i++) ends[i] = jobs[i][0];

    for (int i = 0; i < n; i++) {
        // Binary search: last job that ends <= start of jobs[i]
        int lo = 0, hi = i;
        while (lo < hi) {
            int mid = (lo + hi + 1) / 2;
            if (ends[mid-1] <= jobs[i][1]) lo = mid;
            else hi = mid - 1;
        }
        // Option 1: skip this job. Option 2: take this job
        dp[i+1] = max(dp[i], dp[lo] + jobs[i][2]);
    }
    return dp[n];
}

```

DRY RUN — eraseOverlapIntervals: [[1,2],[2,3],[3,4],[1,3]]

Sorted by end	lastEnd	intervals[i][0] >= lastEnd?	Action	keep
[1,2]	2 (init)	—	init, keep=1	1
[1,3]	2	1 >= 2? NO	SKIP (remove)	1
[2,3]	2	2 >= 2? YES	KEEP, lastEnd=3	2
[3,4]	3	3 >= 3? YES	KEEP, lastEnd=4	3
Result	—	—	total=4, keep=3	remove=4-3=1

GREEDY CORRECTNESS PROOF (EXCHANGE ARGUMENT)

Suppose optimal solution picks interval X that does NOT have the earliest end.

We can SWAP X with the earliest-ending interval Y that doesn't overlap previous picks.

After swap: Y ends no later than X, so future choices are same or better.

Conclusion: earliest-end greedy never does worse than any other choice. QED.

This 'exchange argument' is the standard proof technique for greedy interval scheduling.

VARIANTS COMPARISON TABLE

Problem	Sort By	Greedy Rule	Complexity
Max non-overlapping	End time	Take if start $\geq$ lastEnd	$O(N \log N)$
Min removals	End time	Same as above, count skipped	$O(N \log N)$
Min arrows (balloons)	End time	New arrow if start $>$ arrowPos	$O(N \log N)$
Weighted job scheduling	End time	DP + binary search	$O(N \log N)$
Min meeting rooms	Both start+end	Sweep / priority queue	$O(N \log N)$
Max intervals in K groups	End time	Greedy K times or DP	$O(NK \log N)$

COMMON MISTAKES + EDGE CASES

✗ Sorting by START time for maximum selection — MUST sort by END time

✗ Overlap check: `intervals[i][0]  $\geq$  lastEnd` (use $\geq$, touching is non-overlapping)

✗ Counting removals = total - kept, not just the count of skipped iterations

✗ Arrows problem: same sort-by-end logic, new arrow only when start $>$ arrowPos (not $\geq$)

✗ Weighted scheduling: NOT greedy — must use DP + binary search

INTERVIEW TIPS

✓ Say: 'Sort by end time. Greedy: always pick earliest-finishing non-overlapping interval.'

✓ Explain exchange argument briefly — it proves greedy correctness in 3 sentences

✓ Distinguish: unweighted (greedy $O(N \log N)$) vs weighted (DP + binary search $O(N \log N)$)

✓ Minimum removals = total - max_kept — connect it to the activity selection count

✓ Arrows problem: same algorithm, different framing — notice the structural similarity

CP CONTEST TRICKS

★ Activity selection is the CANONICAL greedy problem — $O(N \log N)$ sort + $O(N)$ greedy

★ Weighted interval scheduling: DP with binary search — $O(N \log N)$, harder but important

★ For K machines/rooms: binary search on K, or greedy with priority queue

★ Fractional scheduling (can split jobs): greedy by deadline (EDF) — $O(N \log N)$

★ Interval scheduling on circle: fix one interval, solve linear problem on rest

SIMILAR PROBLEMS

Problem	Platform	Relation
Non-overlapping Intervals	LeetCode 435	Min removals = total - max kept
Minimum Arrows to Burst Balloons	LeetCode 452	Same sort-by-end greedy
Activity Selection Problem	GFG Classic	Pure activity selection
Job Scheduling with Profit	LeetCode 1235	Weighted: DP + binary search
Meeting Rooms	LeetCode 252	Any overlap? Boolean check
Meeting Rooms II	LeetCode 253	Min rooms = max overlap
Maximum Profit in Job Scheduling	LeetCode 1235	Weighted interval scheduling

Determine if Two Events Have Conflict	LeetCode 2446	Single overlap check
---------------------------------------	---------------	----------------------

BONUS — ADVANCED PATTERN COMBINATIONS

PATTERN COMBINATIONS THAT APPEAR IN HARD PROBLEMS

Hard problems are almost always TWO patterns combined. Recognise the combination, not just the individual.

Combination	Classic Problem	How They Combine	Difficulty
Prefix Sum + HashMap	Subarray Sum Equals K (LC 560)	Prefix builds running sums; HashMap counts how often (sum-k) appeared	Medium
Sort + Two Pointer	3Sum (LC 15), Container With Most Water (LC 11)	Sort + Two Pointer structure; two pointers exploit sorted order in O(N)	Medium
Sliding Window + HashMap	Minimum Window Substring (LC 76)	Window expands/shrinks; HashMap tracks character frequency needed	Hard
Monotonic Stack + Contribution	Sum of Subarray Minimums (LC 907)	Stack finds left/right boundaries; contribution computes total in O(N)	Hard
Binary Search + Greedy	Koko Bananas (LC 875), Split Array (LC 410)	Binary Search on answer; greedy feasibility check in O(N)	Medium
Prefix Sum + Binary Search	Find Pivot Index, Random Pick with Weight	Prefix; binary search finds target range	Medium
Cyclic Sort + Bit Tricks	First Missing Positive (LC 41)	Cyclic sort places elements; scan finds missing	Hard
Frequency Map + Sliding Window	Permutation in String (LC 567)	Fixed window; HashMap compares char frequencies	Medium
Sweep Line + Multiset	The Skyline Problem (LC 218)	Events sorted by x; multiset tracks active heights dynamically	Hard
Difference Array + Prefix Sum	Corporate Flight Bookings (LC 1109)	Diff array O(1) updates; prefix sum reconstructs final array	Medium
Boyer-Moore + Verification Pass	Majority Element I & II (LC 169, 229)	First pass finds candidates; second pass verifies counts	Medium
Coordinate Compress + BIT	Count of Smaller Numbers After Self (LC 315)	Compress large values; BIT answers range frequency queries	Hard
Rolling Hash + Binary Search	Longest Duplicate Substring (LC 1048)	Binary search on length K; rolling hash finds duplicate of length K	Hard
Kadane + Circular Logic	Max Circular Subarray (LC 918)	Standard Kadane for linear; totalSum - minKadane for circular	Medium
Monotonic Deque + DP	Jump Game VI (LC 1696), Constrained Subsequence (LC 1442)	DP over indices; sliding window max/min in deque gives O(1) optimum	Hard

FAANG INTERVIEW QUESTION BANK — MOST ASKED

These exact problems appear most in FAANG loops. Master each one.

Problem	Pattern	Company	Frequency	Must Know
Two Sum	HashMap	Almost everyone	★★★★★	YES
Longest Substring No Repeat	Dynamic Window	Google, Meta, Amazon	★★★★★	YES
Maximum Subarray	Kadane	Google, Amazon, Microsoft	★★★★★	YES
Product Except Self	Prefix/Suffix	Meta, Amazon, Apple	★★★★★	YES
Find Minimum in Rotated Array	Binary Search	Google, Meta, Microsoft	★★★★★	YES
3Sum	Sort+Two Pointer	Google, Meta, Amazon	★★★★★	YES
Container With Most Water	Two Pointer	Google, Meta, Amazon	★★★★★	YES
Merge Intervals	Merge Intervals	Google, Meta, Bloomberg	★★★★★	YES
Sliding Window Maximum	Monotonic Deque	Google, Meta, Amazon	★★★★■	YES
Subarray Sum Equals K	Prefix+HashMap	Google, Facebook, Amazon	★★★★★	YES
Top K Frequent Elements	Freq Count+Heap	Amazon, Google, Meta	★★★★■	YES
Minimum Window Substring	Dynamic Window	Google, Meta, Uber	★★★★■	YES
Group Anagrams	HashMap+Sort	Amazon, Google, Meta	★★★★★	YES
Longest Consecutive Sequence	HashSet	Google, Meta, Amazon	★★★★■	YES
Daily Temperatures	Monotonic Stack	Google, Bloomberg, Uber	★★★★■	YES
Trapping Rain Water	Two Pointer/Stack	Google, Meta, Amazon, MS	★★★★★	YES
Search in Rotated Array	Binary Search	Google, Meta, Amazon	★★★★★	YES
Non-overlapping Intervals	Interval Schedule	Google, Meta, Amazon	★★★★■	YES
Sort Colors	Dutch National Flag	Meta, Microsoft, Amazon	★★★★■	YES
Find Duplicate Number	Floyd/Cyclic Sort	Amazon, Microsoft, Apple	★★★★■	YES
Majority Element	Boyer-Moore	Google, Meta, Amazon	★★★★■	YES
Koko Eating Bananas	BSearch on Answer	Google, Amazon	★★★★■	YES
Jump Game II	Greedy	Google, Meta, Amazon	★★★★■	YES
Largest Rectangle Histogram	Monotonic Stack	Google, Amazon, Bloomberg	★★★★■	YES
Maximum Product Subarray	Kadane Variant	Amazon, Google, Microsoft	★★★★■	YES

INTERVIEW ANTI-PATTERNS — WHAT KILLS YOUR CHANCES

These are the exact mistakes that cost candidates FAANG offers. Study each one.

Anti-Pattern	What Interviewers Think	Fix
Jump straight to code without thinking	Can't structure thoughts, will write wrong code	Always: clarify -> brute force -> optimize -> code
Silent coding (no communication)	Can't explain reasoning, poor team fit	Narrate every decision: 'I'm using hashmap because...'
Wrong complexity analysis	Doesn't understand their own solution	Practice Big-O for every pattern until automatic
Ignoring edge cases	Writes buggy production code	Always say: 'Let me check empty array, single element, all same elements'
Using wrong data structure (map vs unordered_map)	Doesn't know C++ fundamentals	Know: unordered=O(1), ordered=O(log N) — always justify choice
Giving up when stuck	Low problem-solving resilience	Say: 'Let me think about brute force first' — show structured thinking
Not asking clarifying questions	Jumps to conclusions, bad communicator	Always ask: sorted? unique values? integer overflow possible?
Over-engineering simple solutions	Can't distinguish complexity levels	Start simple. Optimize only when interviewer asks
Not testing the solution	Ships untested code	Trace through example manually after writing. Test edge case.
Wrong loop bounds (off-by-one)	Careless, detail-oriented work needed	Dry run mentally: check first/last iteration explicitly
Not knowing STL (using raw arrays)	Unfamiliar with modern C++	Know: vector, unordered_map, stack, deque, priority_queue, set
Panic when interviewer gives hints	Fragile, can't handle feedback	Hints are HELP. Say 'Thanks, let me think about that...'

CODEFORCES SPEED GUIDE — FIRST 30 SECONDS

This is how top CF contestants think when they first read a problem.

Second 0-5	Read constraints	N=?, values=?, time limit=?	This tells you the algorithm
Second 5-10	Classify problem type	Array? String? Graph? Number theory?	Narrows to 3-4 patterns
Second 10-15	Apply constraint table	$N \leq 1e5 \rightarrow O(N \log N)$	Confirms algorithm class
Second 15-20	Check for obvious tricks	Prefix sum? Sorting? XOR?	Often the answer is simple
Second 20-25	Think about edge cases	All same? Empty? N=1?	Prevents WA on first submit
Second 25-30	Start coding	Template ready? Typedef? MOD?	Don't overthink, start typing

COMPETITIVE PROGRAMMING TEMPLATE (C++)

```
#include <bits/stdc++.h>

using namespace std;

// Fast I/O
#define FAST_IO ios_base::sync_with_stdio(false); cin.tie(NULL)

// Common types
typedef long long ll;
typedef vector<int> vi;
typedef vector<ll> vll;
typedef pair<int,int> pii;
typedef vector<pii> vpii;

// Constants
const int MOD = 1e9 + 7;
const int INF = 1e9;
const ll LINF = 1e18;

// Useful macros (use sparingly in interviews)
#define pb push_back
#define F first
#define S second
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()

// Fast power
ll pw(ll base, ll exp, ll mod=MOD) {
    ll res=1; base%=mod;
    while(exp>0){ if(exp&1) res=res*base%mod; base=base*base%mod; exp>>=1; }
    return res;
}

void solve() {
```

```
int n; cin >> n;
// ... solution here
}

int main() {
FAST_IO;
int t = 1;
// cin >> t; // uncomment for multiple test cases
while(t--) solve();
return 0;
}
```

30-DAY MASTERY PLAN — ARRAYS + HASHING

Follow this exactly. One pattern per day with 3 problems. Don't skip days.

Day	Pattern	LeetCode Problems (Easy→Hard)	Goal
1	Arrays + HashMap basics	Two Sum, Contains Duplicate, Valid Anagram	Understand O(N) hash lookup
2	Frequency Count	Top K Frequent, Sort by Frequency, Group Anagrams	Count first, answer later
3	HashSet patterns	Longest Consecutive, Happy Number, Intersection Arrays	Existence O(1)
4	Prefix Sum basics	Range Sum Query, Find Pivot Index, Running Sum	Pre-compute for O(1) query
5	Prefix Sum + HashMap	Subarray Sum=K, Contiguous Array, Binary Subarray Count	sum-k trick mastery
6	Prefix XOR	Single Number I/II/III, Missing Number, XOR Queries	XOR self-inverse
7	Prefix/Suffix Arrays	Product Except Self, Candy, Trapping Rain Water	Two-pass technique
8	Difference Array	Range Addition, Corporate Flights, Car Pooling	O(1) range update
9	Fixed Sliding Window	Moving Average, Max Sum Subarray K, Count Distinct	Add right, remove left
10	Dynamic Sliding Window	Longest No Repeat, Fruit Into Baskets, At Most K Zeros	Template: expand+shrink
11	Min Window / Hard Window	Min Window Substring, Subarrays with K Integers	Minimum window template
12	Two Pointer sorted	Two Sum II, 3Sum, Container Most Water	Sorted array shortcut
13	Two Pointer variants	Remove Duplicates, Trapping Rain, Squares Sorted Array	Multiple pointer styles
14	Dutch National Flag	Sort Colors, Move Zeros, Wiggle Sort	3-way partition
15	Kadane Algorithm	Max Subarray, Best Time Stock, Max Product Subarray	Extend or restart
16	Kadane variants	Circular Subarray Max, Max Abs Sum, K-concat Max Sum	Circular + product
17	Monotonic Stack basics	Daily Temperatures, Next Greater I/II, Stock Span	Stack stores indices
18	Monotonic Stack hard	Largest Rectangle, Maximal Rectangle, Sum Subarray Min	Histogram + contribution
19	Monotonic Queue (Deque)	Sliding Window Max, Jump Game VI, Constrained Sum	Deque = window max/min
20	Contribution Technique	Sum Subarray Min, Sum Subarray Ranges, Total Strength	Flip: element contribution
21	Sorting + Greedy	Non-overlap Intervals, Arrows, Assign Cookies	Sort by end, greedy pick
22	Interval Scheduling	Meeting Rooms II, Merge Intervals, Insert Interval	Activity selection
23	Binary Search basics	Binary Search, Find First/Last, Search Rotated	lo/hi template
24	Binary Search on Answer	Koko Bananas, Ship Packages, Split Array Largest Sum	Feasibility + monotone
25	Boyer-Moore + Cyclic Sort	Majority I/II, Missing Number, Find All Duplicates	O(1) space tricks
26	Bucket + Counting Sort	Top K Bucket, Maximum Gap, H-Index	O(N) sort tricks
27	Sweep Line + Coordinate Compression	Skyline, Falling Squares, Count Smaller After Self	Event-based sweep
28	Bitmasking	Subsets, Shortest Path All Nodes, Smallest Team	Subset = integer
29	Rolling Hash + Modular Math	Rabin-Karp, Longest Dup Substring, Pow(x,n)	Hash + fast power
30	MIXED REVIEW	Random Hard from each chapter (5 problems)	Pattern recognition speed

QUICK RECOGNITION SIGNALS — MASTER INDEX

Read a problem. Spot a signal below. Know the pattern immediately.

You read this in the problem...	Pattern to use	Confidence
'find two numbers that sum to target'	HashMap (Two Sum)	★★★★★
'subarray sum equals K'	Prefix Sum + HashMap	★★★★★
'longest substring without repeating'	Dynamic Sliding Window	★★★★★
'maximum of every window of size K'	Monotonic Deque	★★★★★
'next greater element'	Monotonic Stack	★★★★★
'maximum subarray sum'	Kadane's Algorithm	★★★★★
'product of array except self'	Prefix + Suffix Arrays	★★★★★
'sort array of 0s, 1s, 2s'	Dutch National Flag	★★★★★
'numbers from 1 to N, find missing/duplicate'	Cyclic Sort or XOR	★★★★★
'majority element appears > N/2 times'	Boyer-Moore Voting	★★★★★
'merge overlapping intervals'	Merge Intervals (sort by start)	★★★★★
'max non-overlapping intervals'	Interval Scheduling (sort by end)	★★★★★
'range sum query on static array'	Prefix Sum	★★★★★
'XOR of range [L, R]'	Prefix XOR	★★★★★
'range add X to all elements in [L,R]'	Difference Array	★★★★★
'large values but small N, need as index'	Coordinate Compression	★★★★★
'return answer modulo 10^9+7 '	Modular Arithmetic	★★★★★
'all subsets of N elements ($N \leq 20$)'	Bitmask Enumeration	★★★★★
'find minimum speed/capacity that works'	Binary Search on Answer	★★★★★
'sorted + find pair/triple with sum'	Two Pointer (after sort)	★★★★★
'distinct elements in every window'	Fixed Window + HashMap	★★★★■
'group strings that are anagrams'	HashMap with sorted key	★★★★■
'maximum events active at same time'	Sweep Line	★★★★■
'sum over all subarrays of min/max'	Contribution + Monotonic Stack	★★★★■
'substring matching / repeated pattern'	Rolling Hash / KMP	★★★★■
'N+1 items in N buckets, find gap'	Bucket Sort / Pigeonhole	★★★★■
'count paths / arrangements modulo'	DP + Modular Arithmetic	★★★★■
'count elements less than X in range'	Coordinate Compress + BIT	★★★★■

C++ STL QUICK REFERENCE FOR INTERVIEWS

Know these cold. Wrong STL choice = wrong complexity in interview.

Container	Insert	Delete	Lookup	Use When
vector<T>	O(1) back, O(N) mid	O(1) back, O(N) mid	O(1) index	Default sequence, index access
deque<T>	O(1) front/back	O(1) front/back	O(1) index	Monotonic queue, BFS
stack<T>	O(1) push	O(1) pop	O(1) top	DFS, monotonic stack, undo
queue<T>	O(1) push	O(1) pop	O(1) front	BFS, level-order
priority_queue<T>	O(log N)	O(log N)	O(1) top	Top-K, greedy by priority
unordered_map	O(1) avg	O(1) avg	O(1) avg	Frequency count, lookup
map<K,V>	O(log N)	O(log N)	O(log N)	Sorted keys, order matters
unordered_set	O(1) avg	O(1) avg	O(1) avg	Existence check, dedup
set<T>	O(log N)	O(log N)	O(log N)	Sorted unique, lower_bound
multiset<T>	O(log N)	O(log N)	O(log N)	Sorted with duplicates (Skyline)

Algorithm	Syntax	Complexity	Notes
Binary Search	lower_bound(v.begin(),v.end(),x)	O(log N)	First element >= x
Binary Search	upper_bound(v.begin(),v.end(),x)	O(log N)	First element > x
Sort	sort(v.begin(),v.end())	O(N log N)	Introsort (unstable)
Stable Sort	stable_sort(v.begin(),v.end())	O(N log N)	Preserves equal order
Min/Max	*min_element(v.begin(),v.end())	O(N)	Returns iterator
Accumulate	accumulate(v.begin(),v.end(),0LL)	O(N)	Sum, use 0LL for long long
Count	count(v.begin(),v.end(),x)	O(N)	Count occurrences
Unique	unique(v.begin(),v.end())	O(N)	Remove consecutive dups
Reverse	reverse(v.begin(),v.end())	O(N)	In-place reverse
Next Permutation	next_permutation(v.begin(),v.end())	O(N)	Generate next permutation
Nth Element	nth_element(v.begin(),v.begin()+k,v.end())	O(N) avg	QuickSelect for K-th

QUICK CHEAT SHEET — ALL PATTERNS

Pattern	When to use	Time	Space	Key DS
Frequency Count	Count occurrences, top-K, anagrams	$O(N)$	$O(N)$	unordered_map
HashMap	Two sum, prefix+map, complement check	$O(N)$	$O(N)$	unordered_map
HashSet	Existence check, duplicates, consecutive	$O(N)$	$O(N)$	unordered_set
Prefix Sum	Range sum queries, subarray sum=K	$O(N)+O(1)$	$O(N)$	vector
Prefix XOR	XOR range queries, single/missing num	$O(N)+O(1)$	$O(N)$	vector
Fixed Window	All subarrays of exact size K	$O(N)$	$O(1)$	two ints
Dynamic Window	Longest/shortest satisfying condition	$O(N)$	$O(K)$	map/ints
Two Pointer	Sorted pair/triple sum, palindrome	$O(N)$	$O(1)$	two ints
Kadane	Max subarray sum	$O(N)$	$O(1)$	two ints
Monotonic Stack	Next greater/smaller, histogram	$O(N)$ amort	$O(N)$	stack<int>
Monotonic Queue	Window max/min, k-max constraint	$O(N)$ amort	$O(K)$	deque<int>
Difference Array	Range updates, overlap count	$O(N+Q)$	$O(N)$	vector
Cyclic Sort	Missing/duplicate in $1..N$ range	$O(N)$	$O(1)$	in-place
Sorting+Greedy	Interval scheduling, assignment	$O(N \log N)$	$O(1)$	sort
Boyer-Moore	Majority element ($>N/2$)	$O(N)$	$O(1)$	two vars
Coord Compress	Large values, need index-mapping	$O(N \log N)$	$O(N)$	sort+map
Rolling Hash	Substring match, duplicate substrings	$O(N)$	$O(1)$	long long
Dutch Flag	3-way partition (0,1,2)	$O(N)$	$O(1)$	three ptrs
Contribution	Sum over all subarrays of min/max	$O(N)$	$O(N)$	stack
Sweep Line	Event-based intervals, overlap	$O(N \log N)$	$O(N)$	sort+scan
Binary Search	Monotonic condition on array	$O(N \log N)$	$O(1)$	lo/hi
Bucket Sort	Values in range, $O(N)$ sort	$O(N+K)$	$O(K)$	vector<vector>
Counting Sort	Small integer range, stable sort	$O(N+K)$	$O(K)$	int array
Merge Intervals	Merge overlapping intervals	$O(N \log N)$	$O(N)$	sort+stack
Prefix/Suffix Arr	Product/max/min except self	$O(N)$	$O(N)$	two vectors
Sparse Freq	Offline range frequency with BIT	$O(N \log N)$	$O(N)$	BIT
Bitmask	Subset enumeration, state compress	$O(2^N \cdot N)$	$O(2^N)$	int bitmask
Modular Arith	Large numbers mod prime	$O(N)$	$O(1)$	long long

FAANG FREQUENCY MASTER TABLE

★★★★★ = Extremely Common | ★★★★█ = Very Common | ★★★██ = Common | ★★███ = Less Common

Pattern	Google	Meta	Amazon	Microsoft	Apple	Uber
Frequency Count	★★★★★	★★★★★	★★★★★	★★★★█	★★★★█	★★★★█
HashMap	★★★★★	★★★★★	★★★★★	★★★★★	★★★★★	★★★★★
Prefix Sum	★★★★★	★★★★█	★★★★★	★★★★█	★★★██	★★★★█
Dynamic Window	★★★★★	★★★★★	★★★★█	★★★★★	★★★★█	★★★★★
Two Pointer	★★★★★	★★★★★	★★★★★	★★★★★	★★★★★	★★★★★
Kadane	★★★★★	★★★★█	★★★★★	★★★★★	★★★★█	★★★★█
Monotonic Stack	★★★★★	★★★★★	★★★★█	★★★★█	★★★★█	★★★★█
Sorting+Greedy	★★★★★	★★★★█	★★★★★	★★★★█	★★★★█	★★★★█
Binary Search	★★★★★	★★★★★	★★★★★	★★★★★	★★★★★	★★★★█
Merge Intervals	★★★★█	★★★★★	★★★★★	★★★★█	★★★★█	★★★★★
Cyclic Sort	★★★██	★★★★█	★★★★★	★★★★█	★★★██	★★★██
Boyer-Moore	★★★★█	★★★★█	★★★★█	★★★★█	★★★██	★★★██
Diff Array	★★★██	★★★██	★★★★█	★★★██	★★★██	★★★██
Rolling Hash	★★★★█	★★★★█	★★★██	★★★★█	★★★★█	★★★██

COMMON MISTAKES BIBLE

Read this before every contest/interview. These mistakes cost the most.

Mistake	Pattern	Fix
map instead of unordered_map	All hashing	unordered_map is $O(1)$ avg, map is $O(\log N)$
INT_MIN/INT_MAX overflow in sum	Kadane, prefix sum	Use long long for sums
off-by-one in window size	Sliding window	Window size = right - left + 1
forgetting pre[0]=1 for subarray sum=K	Prefix sum + map	Empty prefix has sum 0, count 1
sorting for wrong field in intervals	Greedy intervals	Sort by END time for max selection
stack stores values not indices	Monotonic stack	Always store INDICES, retrieve values
no verification in Boyer-Moore	Voting	BM gives candidate, verify with second pass
diff[R+1] out of bounds	Difference array	Allocate N+2 size array
nums[i]-1 for 0-indexed cyclic	Cyclic sort	Correct index = nums[i]-1 for 1..N
left < right vs left <= right	Two pointer	Use left < right for pair search
not deduplicating for 3Sum	Two pointer+sort	Skip while nums[i]==nums[i-1]
hash collision in rolling hash	Rolling hash	Always verify by direct comparison
Unicode subscripts in code	Any	Use ASCII only in variable names
no mod in large arithmetic	Modular arith	Apply mod after EVERY multiply
not handling all-negative Kadane	Kadane	Init currMax=nums[0], not 0

INTERVIEW COMMUNICATION GUIDE

What to say at each stage. Practice saying this out loud.

Stage	What to Say	Why It Matters
Read problem	'Let me understand — the input is X, output is Y, constraints say N=1e5.'	Shows systematic thinking
Identify pattern	'The key here is [pattern name] because [signal from problem]'	Demonstrates pattern mastery
Brute force	'Naive approach is O(N²): [explain]. This won't scale for N=1e5.'	Shows you know limits
Optimization	'Observation: [key insight]. This lets me reduce to O(N) by [technique]'	Shows engineering depth
Complexity	'Time O(N log N) for sorting + O(N) for scanning = O(N log N) overall. Space O(1) for the map.'	Proves O(N log N) is optimal
Edge cases	'I should also handle: empty array, all same elements, N=1, overflow.'	Shows thoroughness
Code	Narrate what you're coding: 'I'm using unordered_map here for O(1) lookups because...'	Connects code to logic
Testing	'Let me trace through example 1: ...' then 'edge case: what if array has all zeros?'	Validates solution
Follow-up	'If we needed O(1) space, we could... If input was a stream, we would...'	Signals senior thinking

CP CONTEST PLAYBOOK

30-second pattern identification checklist for Codeforces / competitive contests.

You see this...	→ Try this first	If fails, try
Sum of subarray, count subarrays	Prefix sum + HashMap	Sliding window
Longest/shortest subarray with property	Dynamic sliding window	Two pointer
Next greater / smaller element	Monotonic stack (decreasing/increasing)	Sparse table
Window max/min over all K-windows	Monotonic deque	Segment tree
Array 1..N, find missing/duplicate	Cyclic sort or XOR	Math (sum formula)
Range update, point query	Difference array	Fenwick tree
Majority element	Boyer-Moore ($O(N)$ $O(1)$)	Sort → middle element
Overlapping intervals / max overlap	Sort + sweep / merge intervals	Difference array
Substring matching, hashing strings	Rolling hash (Rabin-Karp)	KMP or Z-algorithm
Large values ($1e9$) as indices	Coordinate compression	HashMap as substitute
Count inversions in array	Merge sort + count	BIT + coordinate compress
Subsets, state with small N (≤ 20)	Bitmask DP	Backtracking
Consecutive / streak problems	HashSet + scan from sequence start	Sort + linear scan
XOR properties, pairs cancel	Prefix XOR / XOR all	Bit manipulation tricks
Pair sum in sorted array	Two pointer ($O(N)$)	Binary search ($O(N \log N)$)
Maximize min / Minimize max	Binary search on answer	Greedy with sorting
Contribution of each element to all subarrays	Contribution technique + stack	DP

PATTERN DEPENDENCY MAP & LEARNING ROADMAP

Learn patterns in this order. Each row builds on the previous.

Level	Patterns to Master	Milestone	Time Estimate
Foundation	Arrays, Strings, HashMaps, HashSets	Two Sum, Valid Anagram, Longest Consecutive	Week 1
Core Level 1	Prefix Sum, Sliding Window (Fixed)	Range Sum, Moving Average	Week 2
Core Level 2	Two Pointer, Dynamic Window, Kadane	3Sum, Min Window Substring, Max Subarray	Week 3
Core Level 3	Monotonic Stack, Sorting+Greedy, Cyclic Sort	Histogram, Interval Scheduling, Missing Number	Week 4
Advanced 1	Prefix XOR, Difference Array, Boyer-Moore	XOR Range Query, Flight Bookings, Majority	Week 5
Advanced 2	Coordinate Compression, Rolling Hash, Sweep Line	2D Range Sum, Rabin-Karp, Skyline	Week 6-7
Expert	Monotonic Queue, Contribution, Sparse Frequency	Sliding Window Max, Sum Subarray Min	Week 8+
Integration	Mix patterns: HashMap+Prefix Sum, Sort+Two Pointer	Subarray Sum=K, 3Sum, Merge Intervals	Ongoing

GOLDEN RULES — NEVER FORGET THESE

01. Read constraints FIRST

$N \leq 1e5 \rightarrow O(N \log N)$, $N \leq 1e6 \rightarrow O(N)$. This alone narrows patterns 80%.

02. Name the pattern before coding

In interviews: say the pattern name + why it fits. In contests: verify it's the right tool.

03. Brute force first, then optimise

Brute force shows WHAT you need. Optimisation shows HOW to get it faster.

04. Prefer unordered_map over map

$O(1)$ amortised vs $O(\log N)$. For $N=1e5$, map with 50 queries = $50 \cdot 17 = 850$ ops vs 50.

05. pre[0]=0 and pre[0]=1 in map

pre[0]=0 for prefix sum array. prefixCount[0]=1 in HashMap for subarray sum=K.

06. Store indices in stacks

Monotonic stack stores INDICES not values — you need both position and value.

07. Verify Boyer-Moore result

Always do a second pass to verify. BM only guarantees a CANDIDATE.

08. Long long for all sums

$N=1e5$, values= $1e9 \rightarrow$ prefix sum can reach $1e14$. int overflows at $\sim 2e9$.

09. Window: right-left+1 for size

NEVER forget the +1. Window [2,5] has size $5-2+1=4$, not 3.

10. Sort by END for interval greedy

Sort by start for merging intervals. Sort by END for selection/scheduling.

11. XOR: pairs cancel, lone survives

XOR all $\rightarrow$ duplicates cancel. This pattern solves single number, missing, etc.

12. Cyclic sort: nums[i]-1 is home

Only for $1..N$. Place $\text{nums}[i]$ at index $\text{nums}[i]-1$. After sort, scan for misfits.

13. Contribution: L*R subarray count

Element at index i has $L=i-\text{leftBoundary}$, $R=\text{rightBoundary}-i$ subarrays.

14. Rolling hash: always verify

Hash equality $\neq$ string equality. Hash collisions happen. Always char-compare on match.

15. Difference array: diff[R+1]=X

ALWAYS mark the END+1, not END. And reconstruct with prefix sum at the end.

You now have the complete Arrays + Hashing arsenal. Don't just read — SOLVE.
One pattern a day, with 3 problems. See you at FAANG. — You've got this.